
UNIVERSIDAD INDUSTRIAL DE SANTANDER

Escuela de Física — Facultad de Ciencias

Doctorado en Física

Integración completa de las mejoras del WCD con soluciones de NaCl en MEIGA

Informe técnico

“Variante MEIGA-WCD-SalphaBeta-2026”

Autor

Jaime A. Betancourt

Doctorado en Física, Universidad Industrial de Santander

13 de septiembre de 2026

Resumen

Este informe documenta la integración, en una única base de código consistente, de las cinco familias de mejoras desarrolladas de forma incremental para el detector Cherenkov de agua (WCD) del framework MEIGA: (1) el registro paso a paso de `G4WCDSSteppingAction`; (2) las densidades y la composición de las soluciones de NaCl al 2.5, 5.0 y 10.0 %; (3) la corrección óptica del recubrimiento de Tyvek (propiedad `RINDEX` en `linerOpticalPT`) junto con las tablas ópticas diferenciadas por concentración; (4) una propuesta alternativa de propiedades ópticas basada en interpolación y dispersión de Rayleigh; y (5) las correcciones de la respuesta del PMT (`G4MPMTAction`, `OptDevice`, material efectivo del fotocátodo). El resultado es un único árbol de código fuente, derivado de la variante `MEIGA-WCD-SalphaBeta-2026`, que conserva el tratamiento térmico $S(\alpha, \beta)$ del hidrógeno del agua y añade, de forma consistente, todas las correcciones ópticas y de respuesta del PMT revisadas en los informes técnicos previos. Se documentan también las inconsistencias encontradas en los archivos de entrada (una rama óptica alternativa incompatible con la finalmente adoptada, un archivo con nombre incorrecto, y un archivo huérfano con nombre corrupto) y la manera en que se resolvieron.

Índice

1	Objeto y alcance del informe	2
2	Archivo base y archivos de entrada	2
2.1	Base: <i>Meiga anterior</i>	2
2.2	Archivos de mejora aportados	2
3	Metodología de integración	3
3.1	Hallazgos durante la integración	4
4	Resumen de archivos modificados	4
5	Descripción detallada de los cambios por grupo	5
5.1	Grupo 1 — Registro paso a paso: <code>G4WCDSSteppingAction</code>	5
5.2	Grupo 2 — Densidades y composición de las soluciones de NaCl	5
5.3	Grupo 3 — Corrección óptica del Tyvek y tablas por concentración	5
5.4	Grupo 4 — Rama óptica alternativa (no integrada)	6
5.5	Grupo 5 — Correcciones de la respuesta del PMT	6
5.6	Reintegración del modelo térmico $S(\alpha, \beta)$	7
6	Validación realizada y pendiente	7
7	Archivos entregados	7
8	Conclusiones	8
A	Diffs unificados: Meiga anterior vs. versión integrada	8
A.1	<code>Applications/G4WCDSimulator/G4WCDSSteppingAction.h</code>	8
A.2	<code>Applications/G4WCDSimulator/G4WCDSSteppingAction.cc</code>	9
A.3	<code>G4Models/Materials.h</code>	19
A.4	<code>G4Models/Materials.cc</code>	20
A.5	<code>G4Models/SaltyWCD.cc</code>	26
A.6	<code>G4Models/G4MPMTAction.h</code>	36
A.7	<code>G4Models/G4MPMTAction.cc</code>	37
A.8	<code>Framework/Detector/OptDevice.cc</code>	38
B	Archivos de referencia sin cambios	40

1. Objeto y alcance del informe

El propósito de este documento es dejar constancia completa, reproducible y auditable de los cambios de código fuente realizados sobre la variante **MEIGA-WCD-SalpaBeta-2026** del framework MEIGA para incorporar las cinco familias de mejoras desarrolladas para el estudio del WCD con agua y soluciones de NaCl. El informe:

- identifica el archivo base (*Meiga anterior*) sobre el cual se aplicaron los cambios;
- describe la metodología usada para reconstruir, a partir de once archivos ***-mejorado/*-corregido** parcialmente superpuestos, una única versión consistente de cada archivo fuente;
- documenta cada cambio, archivo por archivo, con su motivación física o de ingeniería;
- reporta explícitamente los hallazgos que requirieron una decisión de integración (rama óptica alternativa no incorporada, archivo mal etiquetado, archivo huérfano); y
- incluye, en el Apéndice A, el *diff* unificado completo (archivo original de MEIGA contra archivo final integrado) de cada uno de los ocho archivos modificados.

No se recompiló el simulador dentro de este entorno de trabajo porque no hay una instalación de Geant4 disponible; la validación realizada aquí es estática (revisión de consistencia de símbolos, balance de llaves y paréntesis, y verificación cruzada de todo el árbol **src/** en busca de referencias rotas). La Sección 6 detalla el alcance exacto de esta verificación y lo que queda pendiente en la máquina de simulación.

2. Archivo base y archivos de entrada

2.1. Base: *Meiga anterior*

El punto de partida es el árbol de código fuente contenido en **MEIGA-WCD-SalpaBeta-2026.zip**, una variante independiente de MEIGA que ya incorpora el tratamiento térmico $S(\alpha, \beta)$ del hidrógeno ligado del agua (elemento **TS_H_of_Water**, registro de **G4ThermalNeutrons** sobre **QGSP_BERT_HP** en **G4MPhysicsList.cc**, y las cuatro densidades de referencia a 20 °C: 0.99820, 1.01604, 1.03401 y 1.07051 g/cm³ para 0, 2.5, 5 y 10 % de NaCl). Este es el árbol referido como *Meiga anterior* en el resto del informe.

2.2. Archivos de mejora aportados

Se recibieron once archivos fuente ***-mejorado/*-corregido** distribuidos en cuatro carpetas de trabajo, además de sus informes técnicos en L^AT_EX. La Tabla 1 resume su procedencia y su relación de dependencia, reconstruida a partir de las marcas de tiempo del sistema de archivos y de un análisis diferencial exhaustivo entre cada par de versiones sucesivas.

Archivo	Grupo	Contenido / relación
G4WCDSteppingAction-m ejorado.h/.cc	1	Reescritura completa e independiente.
Materials-mejorado.cc, SaltyWCD-mejorado.cc	2	Primera revisión: densidades a 20 °C y las tres soluciones discretas de NaCl. Base de las revisiones 3 y 4.

Archivo	Grupo	Contenido / relación
Materials-mejorado-optica-Tyveck.cc, SaltyWCD-mejorado-optica-Tyveck.cc	3a	Deriva de la revisión 2: tabla óptica RINDEX/ABSLENGTH/MIEHG propia por concentración.
Materials-mejorado-optica-Tyvek-RINDEX.cc, SaltyWCD-mejorado-optica-Tyvek-RINDEX.cc	3b	Deriva de 3a: añade RINDEX a <code>linerOpticalPT</code> y cambia la superficie del Tyvek a <code>dielectric_dielectric/groundbackpainted</code> .
Materials-mejorado-optica.cc, SaltyWCD-mejorado-optica.cc	4	Deriva de la revisión 2, en una rama independiente de 3a/3b: modelo de interpolación normalizado al índice en la línea D del sodio, con dispersión de Rayleigh. Cronológicamente anterior a 3a y no continuada .
Materials-corregido.cc, SaltyWCD-corregido.cc	5	Deriva de 3b: añade las correcciones de material efectivo del PMT (Pyrex) y la geometría por capas explícitas del WCD. Versión final de <code>Materials.cc/SaltyWCD.cc</code> .
G4MPMAction-mejorado.h/.cc	5	Activación en <code>fGeomBoundary</code> , terminación del fotón, deduplicación de fotoelectrones.
OptDevice-mejorado.cc	5	Interpolación de la eficiencia cuántica R5912 (300–650 nm), remoción de archivos de depuración por fotón.
Materials.h (en Correcciones-PMT/)	—	Hallazgo: el contenido de este archivo es en realidad una copia intermedia de <code>SaltyWCD::BuildDetector</code> , no un encabezado. Ver Sección 3.1.
Detector.h/.cc, OptDevice.h, OpticalPhysics.cc/.hh, G4MPMAction.h/.cc (sin sufijo, en Correcciones-PMT/)	—	Copias de referencia, idénticas byte a byte al Meiga anterior. No requieren ninguna acción.

Cuadro 1: Procedencia y relación de dependencia de los archivos de mejora recibidos.

3. Metodología de integración

La integración se realizó en cuatro pasos:

1. **Reconstrucción del árbol de dependencias.** Se calculó el *hash* MD5 de cada archivo para detectar duplicados exactos entre carpetas (por ejemplo, `G4WCDSteppingAction-mejorado.cc` aparece igual en la carpeta raíz y en `Correcciones-PMT/`), y se generó el *diff* unificado entre cada par de versiones sucesivas de `Materials.cc` y `SaltyWCD.cc` para determinar qué archivo es una extensión directa de cuál otro (Tabla 1).
2. **Selección de la rama a integrar.** Cuando dos archivos resultaron ser implementaciones alternativas e incompatibles de la misma funcionalidad (grupo 3 frente a grupo 4, ambos sobre propiedades ópticas diferenciadas por concentración de NaCl), se adoptó la rama que efectivamente continuó hacia la versión corregida final (grupo 3 → grupo 5), y se documentó la rama no adoptada (Sección 5.4).

3. **Reintegración del tratamiento $S(\alpha, \beta)$.** Se verificó, comparando contra el Meiga anterior, que la cadena de archivos de mejora fue desarrollada a partir de una copia de `Materials.cc` *sin* el elemento `TS_H_of_Water`. Se restituyó este elemento y su uso en `Water` y en las tres soluciones salinas, de modo que el simulador final conserve el modelo térmico ya validado en el Meiga anterior (Sección 5.6).
4. **Verificación de consistencia cruzada.** Se recorrió todo `src/` en busca de referencias a los símbolos modificados (`Materials::SaltyWater`, `Materials::HDPE`, `Materials::Pyrex`, `Materials::LinerOptSurf2`, nombres de materiales usados en búsquedas por cadena, etc.) para confirmar que ningún otro modelo de detector (`WCD.cc`, `Hodoscope.cc`, `Mudulus.cc`, `Musaic.cc`, `Scintillator.cc`) ni ninguna aplicación quedara con una referencia rota.

3.1. Hallazgos durante la integración

- **Archivo Correcciones-PMT/Materials.h mal etiquetado.** Su contenido (424 líneas) no corresponde a un encabezado de clase: es, byte a byte salvo una única diferencia menor, una versión intermedia de `SaltyWCD::BuildDetector` (idéntica en un 99 % a `SaltyWCD-corregido.cc`). No se usó como fuente de cambios para `Materials.h`; en su lugar, las declaraciones necesarias para el nuevo encabezado (`SaltyWater_2p5`, `SaltyWater_5`, `SaltyWater_10`) se derivaron directamente de los símbolos nuevos que `Materials-corregido.cc` efectivamente define y usa.
- **Archivo huérfano con nombre corrupto.** El paquete `MEIGA-WCD-SalphiBeta-2026.zip` contenía, en `src/G4Models/`, un archivo cuyo nombre es el carácter suelto «ç» (sin extensión). Su contenido es un borrador antiguo de `SaltyWCD::BuildDetector` anterior incluso a la introducción del modelo $S(\alpha, \beta)$ (usa `newG4Material("SaltyWater", 1.1*g/cm3, 2)`, sin relación con las densidades a 20 °C documentadas en el `CHANGELOG`). Al no tener extensión `.cc`, el sistema de compilación (`file(GLOBsources*.cc)` en `CMakeLists.txt`) nunca lo incluyó en la compilación, por lo que es inerte pero confuso. Se eliminó del árbol final.
- **Rama óptica alternativa (grupo 4) no continuada.** Ver Sección 5.4.

4. Resumen de archivos modificados

Archivo (ruta relativa a <code>src/</code>)	Líneas orig.	Líneas final	Grupo(s)
<code>Applications/G4WCDSimulator/G4WCDSupport/eppingAction.h</code>	44	33	1
<code>Applications/G4WCDSimulator/G4WCDSupport/eppingAction.cc</code>	398	338	1
<code>G4Models/Materials.h</code>	115	120	2, 3, 5
<code>G4Models/Materials.cc</code>	654	875	2, 3, 5 (+ $S(\alpha, \beta)$)
<code>G4Models/SaltyWCD.h</code>	34	34	sin cambios
<code>G4Models/SaltyWCD.cc</code>	258	466	2, 3, 5
<code>G4Models/G4MPMTAction.h</code>	37	38	5
<code>G4Models/G4MPMTAction.cc</code>	141	144	5
<code>Framework/Detector/OptDevice.h</code>	155	155	sin cambios
<code>Framework/Detector/OptDevice.cc</code>	363	341	5

Cuadro 2: Archivos de código fuente afectados por la integración.

5. Descripción detallada de los cambios por grupo

5.1. Grupo 1 — Registro paso a paso: `G4WCDDSteppingAction`

La clase se reescribió por completo manteniendo su interfaz pública (constructor y `UserSteppingAction`). Los cambios principales son:

- **Unidades explícitas y normalizadas:** todas las longitudes se expresan en cm y las energías en MeV o eV según corresponda, eliminando la mezcla de unidades del Meiga anterior.
- **Identificadores de corrida y evento:** cada fila TSV incluye `run_id` y `event_id`, obtenidos de `G4RunManager::GetCurrentRun()/GetCurrentEvent()`.
- **Identificación correcta del núcleo capturador:** en vez de leer `material->GetElement(0)->GetName()` (que en una mezcla reporta arbitrariamente el primer elemento declarado, no el que realmente participó en la reacción), se usa `G4HadronicProcess::GetTargetNucleus()` para obtener el (Z, A) real del núcleo capturador.
- **Separación primarias/secundarias:** las tablas `neutrones-incidentes.tsv`, `secundarias-captura-\protect\penalty\z@neutron.tsv`, `secundarias-inelastica-neutron.tsv`, `secundarias-interaccion.tsv` y `fotones-cherenkov.tsv` separan explícitamente el registro de la partícula primaria (paso 1, `parentID==0`) del de las partículas secundarias generadas en cada interacción.
- **Escritura seguridad en multihilo:** `AppendTsv` usa un `G4Mutex` (`G4AutoLock`) para serializar la escritura concurrente de varios hilos de Geant4 sobre el mismo archivo TSV, y escribe el encabezado una única vez.
- **Formato:** los antiguos archivos `.txt` de columnas separadas por tabulador sin encabezado se reemplazan por tablas `.tsv` con una línea de encabezado explícita (`stepHeader`, `secondaryHeader`, `captureHeader`).

5.2. Grupo 2 — Densidades y composición de las soluciones de NaCl

Se añaden a `Materials` tres materiales discretos, `SaltyWater_2p5`, `SaltyWater_5` y `SaltyWater_10`, contruidos por `CreateSaltyWaterByFinalMassFraction` a partir de la fracción másica final $w_{\text{NaCl}} = m_{\text{NaCl}} / (m_{\text{H}_2\text{O}} + m_{\text{NaCl}})$, con las densidades a 20 °C ya validadas en el Meiga anterior (0.99820, 1.01604, 1.03401 y 1.07051 g/cm³). El material anterior `Materials::SaltyWater` (una única mezcla continua construida dinámicamente en `SaltyWCD::BuildDetector` para cualquier fracción de NaCl) se conserva como alias de compatibilidad hacia `SaltyWater_10`, pero `SaltyWCD::BuildDetector` ahora selecciona explícitamente uno de los cuatro medios predefinidos (`Water`, `SaltyWater_2p5/5/10`) mediante la función `SelectWCDMedium`, que aborta la ejecución con `G4Exception` si se solicita una concentración no tabulada, en lugar de aproximarla silenciosamente. También se corrigen, en esta misma revisión, tres propiedades ópticas del Tyvek que estaban físicamente mal definidas en el Meiga anterior: la longitud de absorción de volumen pasa de 10 m (transparente) a 0.1 mm (opaco, consistente con un reflector difuso real), el índice de refracción de volumen pasa de 0 a 1.49, y el `BACKSCATTERCONSTANT` pasa de 0 a 0.02.

5.3. Grupo 3 — Corrección óptica del Tyvek y tablas por concentración

Esta revisión reemplaza la única tabla óptica compartida `waterPT1/waterPT2` (originalmente un índice de refracción constante de dos puntos, 1.33 en todo el espectro) por una tabla completa e independiente por medio activo (`CreateWCDOpticalTable`), cada una con su propio `RINDEX` espectral de 32 puntos (agua pura y las tres soluciones), una `ABSLNGTH` común como línea base, y el modelo Mie del ejemplo `OpNovice` de Geant4 conservado explícitamente como aproximación declarada. Adicionalmente, se corrige `linerOpticalPT`: se le añade

la propiedad `RINDEX` (requerida por `G4OpBoundaryProcess` para una superficie `dielectric_dielectric`), y la superficie `LinerOptSurf` pasa de `dielectric_metal/ground` a `dielectric_dielectric/groundbackpainted`, el modelo correcto para un recubrimiento reflectante sobre un medio dieléctrico (agua) en vez de un metal.

5.4. Grupo 4 — Rama óptica alternativa (no integrada)

`Materials-mejorado-optica.cc` y `SaltyWCD-mejorado-optica.cc` proponen un modelo distinto para la misma necesidad física (diferenciar las propiedades ópticas del agua pura y las soluciones de NaCl): en vez de tablas `RINDEX` medidas punto a punto, interpolan la curva de dispersión del agua pura y la desplazan verticalmente hasta que su valor en la línea D del sodio (589.3 nm) coincide con un índice objetivo por concentración; añaden además dispersión de Rayleigh explícita (`RAYLEIGH`) con una longitud de referencia de 100 m a 400 nm.

Un análisis cronológico de las marcas de tiempo del sistema de archivos muestra que esta propuesta se escribió *antes* que la de tablas tabuladas del grupo 3 (15:11–15:28 frente a 15:45–16:32 del 2026-09-11), y que la cadena que efectivamente continuó hacia la revisión final `-corregido` (con las correcciones de PMT) es la del grupo 3, no esta. Como ambas modifican las mismas tablas de propiedades del mismo material de forma mutuamente excluyente, **no es posible aplicar ambas simultáneamente sin reintroducir una inconsistencia física** (¿qué `RINDEX` usa el agua pura?). Se optó por integrar la rama efectivamente continuada (grupo 3), dejando esta propuesta documentada aquí como una alternativa evaluada y no adoptada en la versión final del simulador. Su archivo fuente se conserva sin cambios en la carpeta de insumos por si se desea retomar como línea de trabajo futura o como comparación de sistemáticas.

5.5. Grupo 5 — Correcciones de la respuesta del PMT

- **`G4MPMTAction::ProcessHits`**: la respuesta del PMT ahora se activa únicamente cuando el fotón óptico cruza la frontera geométrica de entrada (`step->GetPreStepPoint()->GetStepStatus()==fGeomBoundary`); los fotones aceptados, rechazados por rango de energía y rechazados por eficiencia cuántica se terminan explícitamente (`track->SetTrackStatus(fStopAndKill)`), evitando que el mismo fotón genere más de un fotoelectrón en pasos repetidos dentro del volumen de Pyrex.
- **`OptDevice::GetQuantumEfficiency`**: la eficiencia cuántica nominal del PMT R5912 se interpola linealmente entre ocho puntos tabulados de 300 a 650 nm, en vez de usar una función escalón de 50 nm con saltos artificiales. La eficiencia de colección se aplica una única vez, sobre el resultado de la interpolación.
- **Eliminación de archivos de depuración por fotón**: se retiran las escrituras a `eficiencia-limpia*.txt` dentro de `IsPhotonDetected/GetQuantumEfficiency`, que generaban un archivo de una línea por cada fotón óptico simulado.
- **Material efectivo del PMT (Pyrex)**: el rango de energía de su tabla óptica pasa de un intervalo fijo (2.00–4.00 eV) a exactamente el mismo rango que `OptDevice` usa para la eficiencia cuántica (300–650 nm, expresado en eV), y se documenta explícitamente que el fotocátodo es un absorbente efectivo: la eficiencia cuántica y de colección se muestrean una única vez en `G4MPMTAction/OptDevice`, no en la tabla de propiedades del material.
- **Geometría del WCD (`SaltyWCD::BuildDetector`)**: se reescribe con capas explícitas y no superpuestas (agua/salmuera, Tyvek superior/inferior/lateral, acero superior/inferior/-lateral) en vez del anidamiento de *casings* del Meiga anterior, y se añaden verificaciones con `G4Exception` que abortan la construcción de la geometría si el medio activo no tiene

RINDEX/ABSLENGTH, si la superficie del Tyvek no tiene RINDEX/REFLECTIVITY, o si el material efectivo del PMT no tiene RINDEX/ABSLENGTH.

5.6. Reintegración del modelo térmico $S(\alpha, \beta)$

Los cinco grupos de mejora se desarrollaron, según pudo determinarse comparando `Materials-mejorado.cc` contra el Meiga anterior, a partir de una copia de `Materials.cc` *previa* a la introducción del elemento `TS_H_of_Water`: construían `Water` con `elH` (hidrógeno libre) y densidad nominal de Geant4 (1.0 g/cm^3), en vez de con el elemento ligado y la densidad medida a 20°C (0.99820 g/cm^3). Aplicar esa cadena de cambios tal cual habría eliminado silenciosamente el tratamiento térmico $S(\alpha, \beta)$ ya validado en el Meiga anterior –el objeto central de la variante `Salphabeta-2026`– para *todas* las soluciones de NaCl y para el agua pura.

En la integración final se restituyó `elTSHWater` en `Materials::CreateElements`, se reconstruyó `Water` con el mismo estado termodinámico explícito del Meiga anterior (`kStateLiquid`, 293.15 K , 1 bar) usando `elTSHWater` en vez de `elH`, y se propagó el mismo elemento ligado a los tres materiales salinos nuevos del grupo 2 (`CreateSaltyWaterByFinalMassFraction` ahora recibe `elTSHWater` como elemento de hidrógeno, y construye cada material con el mismo estado termodinámico). De esta manera, `G4ThermalNeutrons` –que se activa automáticamente sobre cualquier material que contenga un elemento `TS_*` reconocido, sin necesidad de cambios en `G4MPhysicsList.cc`– aplica la ley $S(\alpha, \beta)$ del agua de forma consistente a los cuatro medios activos del WCD (agua pura y las tres soluciones), tal como especifica el CHANGELOG original de la variante `Salphabeta`: “*El H de las soluciones reutiliza la evaluación `h_water`; Na, Cl y O siguen el tratamiento estándar de G4NDL*”. Los apéndices de la Sección A muestran este parche como parte del *diff* de `Materials.cc`.

6. Validación realizada y pendiente

- **Realizada en este entorno** (sin instalación de Geant4 disponible): verificación de balance de llaves y paréntesis en cada archivo modificado; búsqueda exhaustiva, en todo `src/`, de referencias a cada símbolo público añadido, renombrado o eliminado de `Materials`; confirmación de que `WCD.cc`, `Hodoscope.cc`, `Mudulus.cc`, `Musaic.cc` y `Scintillator.cc` —que comparten la instancia *singleton* de `Materials`— siguen compilando lógicamente contra la nueva interfaz; confirmación de que `G4WCDConstruction.cc` invoca `SaltyWCD::BuildDetector` con la misma firma.
- **Pendiente, a realizar en la máquina de simulación con Geant4 10.7.3 y G4NDL 4.6 instalados**: compilación completa (`cmake&&make`), una corrida corta de verificación por cada una de las cuatro concentraciones de NaCl, e inspección de las nuevas tablas TSV generadas por `G4WCDSteppingAction` para confirmar que `capturas-neutrones.tsv` reporta $Z = 1$ para el hidrógeno y que la fracción de capturas por debajo de 4 eV es consistente con la obtenida en la validación original de la variante `Salphabeta`.

7. Archivos entregados

Junto con este informe se entrega:

- El árbol de código fuente completo de MEIGA con todas las mejoras integradas (`MEIGA-final/`), listo para compilar en la máquina de simulación.
- Copia de los ocho archivos originales del Meiga anterior que fueron modificados (carpeta `originales/`), y copia de su versión final integrada (carpeta `finales/`), para inspección directa lado a lado.

- El *diff* unificado de cada uno de los ocho archivos (carpeta `diffs/` y Apéndice A de este informe).

8. Conclusiones

Las cinco familias de mejoras revisadas en los informes técnicos previos –registro paso a paso, densidades y composición de las soluciones de NaCl, corrección óptica del Tyvek con tablas por concentración, correcciones de la respuesta del PMT, y una propuesta óptica alternativa– se integraron en una única base de código consistente derivada del Meiga anterior. La integración requirió resolver tres inconsistencias no triviales en los archivos de entrada (una rama óptica alternativa incompatible, un archivo con nombre incorrecto, y un archivo huérfano de nombre corrupto) y restituir el tratamiento térmico $S(\alpha, \beta)$ que la cadena de archivos de mejora había perdido por partir de una copia anterior a su introducción. El resultado conserva íntegramente el objetivo original de la variante **SalphaBeta-2026** –dispersión térmica realista del hidrógeno del agua y las soluciones de NaCl– y añade sobre él todas las correcciones ópticas y de respuesta del PMT revisadas, dejando el simulador listo para compilar y validar en la máquina de simulación.

A. Diffs unificados: Meiga anterior vs. versión integrada

Cada listado muestra el *diff* unificado (`diff-u`) entre el archivo original del Meiga anterior (---) y la versión final integrada en este trabajo (+++). Las líneas que comienzan con `-` se eliminaron; las que comienzan con `+` se añadieron.

A.1. Applications/G4WCDSimulator/G4WCDDSteppingAction.h

```

1  --- MEIGA-anterior/src/Applications/G4WCDSimulator/G4WCDDSteppingAction.h
2  +++ MEIGA-integrado/src/Applications/G4WCDSimulator/G4WCDDSteppingAction.h
3  @@ -1,44 +1,33 @@
4  -// definition of the G4WCDDSteppingAction class
5  +// Registro de informacion paso a paso del modelo WCD.
6
7  #ifndef G4WCDDSteppingAction_h
8  #define G4WCDDSteppingAction_h 1
9  -#include <iostream>
10 +
11 #include "G4UserSteppingAction.hh"
12 -#include "G4String.hh"
13
14 #include "Event.h"
15
16 -
17 +class G4Step;
18 class G4WCDConstruction;
19 class G4WCDEventAction;
20
21 -/* Stepping action class.
22 -   UserSteppingAction collects info at a step level
23 -*/
24 -
25 class G4WCDDSteppingAction : public G4UserSteppingAction
26 {
27 - public:
28     G4WCDDSteppingAction(const G4WCDConstruction* det, G4WCDEventAction* event, Event& theEvent);
29     virtual ~G4WCDDSteppingAction();
30     virtual void UserSteppingAction(const G4Step* step);
31
32 - private:
33     const G4WCDConstruction* fDetectorConstruction;
34     G4WCDEventAction* fEventAction;
35     std::ofstream* fOutputFile;
36     Event& fEvent;
37
38     G4int eventCounter; // Contador de eventos

```

```

39 -      G4int previousEventID; // Almacena el ID del evento anterior
40 -
41 -
42 -      G4double fSiPMTTime;
43 -      G4double fScinTime;
44 -      G4String stepVolume;
45 -      G4String trackVolume;
46 -
47 - std::map<G4String, G4int> fReactionsPerEvent; // Mapa para registrar las reacciones
48 + public:
49 +   G4WCDSSteppingAction(const G4WCDConstruction* detector,
50 +                       G4WCDEventAction* eventAction,
51 +                       Event& event);
52 +
53 +   ~G4WCDSSteppingAction() override = default;
54 +
55 +   void UserSteppingAction(const G4Step* step) override;
56 +
57 + private:
58 +   // Se conservan estas referencias para mantener la interfaz de MEIGA y
59 +   // permitir que la clase se amplie posteriormente sin cambiar su constructor.
60 +   const G4WCDConstruction* fDetectorConstruction;
61 +   G4WCDEventAction* fEventAction;
62 +   Event& fEvent;
63 };
64
65 #endif

```

A.2. Applications/G4WCDSimulator/G4WCDSSteppingAction.cc

```

1 --- MEIGA-anterior/src/Applications/G4WCDSimulator/G4WCDSSteppingAction.cc
2 +++ MEIGA-integrado/src/Applications/G4WCDSimulator/G4WCDSSteppingAction.cc
3 @@ -1,398 +1,338 @@
4 // implementation of the G4WCDSSteppingAction class
5 -#include "G4WCDSimulator.h"
6 /// Registro de informacion paso a paso del modelo WCD.
7 +
8 + #include "G4WCDSSteppingAction.h"
9 -#include "G4WCDConstruction.h"
10 -#include "G4WCDEventAction.h"
11 -#include "G4WCDTrackingAction.h"
12
13 +#include "G4AutoLock.hh"
14 +#include "G4DynamicParticle.hh"
15 +#include "G4Element.hh"
16 +#include "G4Event.hh"
17 +#include "G4Exception.hh"
18 +#include "G4HadronicProcess.hh"
19 +#include "G4Material.hh"
20 +#include "G4Nucleus.hh"
21 +#include "G4OpticalPhoton.hh"
22 +#include "G4ParticleDefinition.hh"
23 +#include "G4Run.hh"
24 +#include "G4RunManager.hh"
25 + #include "G4Step.hh"
26 +#include "G4StepPoint.hh"
27 +#include "G4SystemOfUnits.hh"
28 + #include "G4Track.hh"
29 -#include "G4RunManager.hh"
30 +#include "G4VProcess.hh"
31
32 -#include "G4OpticalPhoton.hh"
33 +#include <fstream>
34 +#include <iomanip>
35 +#include <sstream>
36 +#include <string>
37 +#include <vector>
38
39 -#include "SimData.h"
40 -#include "Detector.h"
41 +namespace
42 +{
43 +G4Mutex outputMutex = G4MUTEX_INITIALIZER;

```

```

44 +
45 +const G4String outputDirectory = "Datos-simulacion/";
46 +
47 +const char* stepHeader =
48 + "# run_id\tevent_id\tparticle\track_id\tparent_id\step_number\t"
49 + "creator_process\step_process\material\tpre_x_cm\tpre_y_cm\tpre_z_cm\t"
50 + "post_x_cm\tpost_y_cm\tpost_z_cm\tpre_energy_MeV\tpost_energy_MeV\t"
51 + "global_time_ns";
52 +
53 +const char* secondaryHeader =
54 + "# run_id\tevent_id\category\tparent_particle\tparent_track_id\t"
55 + "parent_step_number\tsecondary_index\tparticle\track_id\tparent_id\t"
56 + "creator_process\material\tx_cm\ty_cm\tz_cm\kinetic_energy_MeV\t"
57 + "global_time_ns";
58 +
59 +const char* captureHeader =
60 + "# run_id\tevent_id\tparticle\track_id\tparent_id\step_number\t"
61 + "step_process\material\ttarget_Z\ttarget_A\tpre_x_cm\tpre_y_cm\tpre_z_cm\t"
62 + "post_x_cm\tpost_y_cm\tpost_z_cm\tpre_energy_MeV\tpost_energy_MeV\t"
63 + "global_time_ns";
64
65 -G4WCDDSteppingAction::G4WCDDSteppingAction(const G4WCDDConstruction* det, G4WCDEventAction* G4event, Event&
66   theEvent)
67 - : G4UserSteppingAction(),
68 - fDetectorConstruction(det),
69 - fEventAction(G4event),
70 - fEvent(theEvent)
71 -{
72 -
73 - // Inicializa el mapa para registrar las reacciones
74 - fReactionsPerEvent.clear();
75 -}
76
77 -G4WCDDSteppingAction::~G4WCDDSteppingAction()
78 +G4String ProcessName(const G4VProcess* process)
79 +{
80 + return process ? process->GetProcessName() : "none";
81 +}
82
83 -void
84 -G4WCDDSteppingAction::UserSteppingAction(const G4Step* step)
85 -{
86 - G4double paso = step->GetTrack()->GetCurrentStepNumber();
87 - G4Track* track = step->GetTrack();
88 - G4String particleName = track->GetParticleDefinition()->GetParticleName();
89 - G4StepPoint* preStepPoint = step->GetPreStepPoint();
90 - G4ThreeVector position_i = preStepPoint->GetPosition();
91 - G4double x_i = position_i.x(); // Coordenada x
92 - G4double y_i = position_i.y(); // Coordenada y
93 - G4double z_i = position_i.z(); // Coordenada z
94 - G4double E_i = preStepPoint->GetKineticEnergy();
95 - G4StepPoint* postStepPoint = step->GetPostStepPoint();
96 - G4ThreeVector position_f = postStepPoint->GetPosition();
97 - G4double x_f = position_f.x(); // Coordenada x
98 - G4double y_f = position_f.y(); // Coordenada y
99 - G4double z_f = position_f.z(); // Coordenada z
100 - G4double E_f = postStepPoint->GetKineticEnergy();
101 - G4String processName = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName();
102 - G4double parentID = step->GetTrack()->GetParentID();
103 - G4int trackID = track->GetTrackID();
104 -
105 - // Diagnostic stream for the S(alpha,beta) validation. Geant4 reports the
106 - // process as hadElastic for both free-gas and thermal scattering, so the
107 - // material name, temperature and energy transfer are recorded explicitly.
108 - if (particleName == "neutron" && processName == "hadElastic" &&
109 -     E_i <= 4.0*eV) {
110 -     const G4Material* thermalMaterial = preStepPoint->GetMaterial();
111 -     std::ofstream thermalOutput(
112 -         "Datos-simulacion/dispersion-termica-SalphaBeta.tsv",
113 -         std::ios_base::app);
114 -     thermalOutput
115 -         << trackID << '\t'
116 -         << paso << '\t'
117 -         << thermalMaterial->GetName() << '\t'

```

```

117 -         << thermalMaterial->GetTemperature()/kelvin << '\t'
118 -         << E_i/eV << '\t'
119 -         << E_f/eV << '\t'
120 -         << (E_f-E_i)/eV << '\n';
121 -     }
122 -
123 +G4String MaterialName(const G4StepPoint* point)
124 +{
125 +     return (point && point->GetMaterial()) ? point->GetMaterial()->GetName()
126 +         : "none";
127 +}
128 -
129 -     std::ofstream outputFileinformacion("Datos-simulacion/informacion-completa.txt", std::ios_base::app);
130 -     outputFileinformacion << paso << "\t" << particleName << "\t" << trackID << "\t" << parentID << "\t" << E_i
131 -         / MeV << "\t" << x_i << "\t" << y_i << "\t" << z_i << "\t" << x_f << "\t" << y_f << "\t" << z_f << "\t" << E_f / MeV
132 -         << "\t" << postStepPoint->GetProcessDefinedStep()->GetProcessName() << "\n";
133 -     outputFileinformacion.close();
134 -
135 -     if(paso == 1 && particleName == "neutron" )
136 -     { std::ofstream outputFileisertados("Datos-simulacion/neutrones-incertados.txt", std::ios_base::app);
137 -     outputFileisertados << paso << "\t" << particleName << "\t" << trackID << "\t" << parentID << "\t" << E_i /
138 -         MeV << "\t" << x_i << "\t" << y_i << "\t" << z_i << "\t" << x_f << "\t" << y_f << "\t" << z_f << "\t" << E_f / MeV <<
139 -         "\t" << postStepPoint->GetProcessDefinedStep()->GetProcessName() << "\n";
140 -     outputFileisertados.close();}
141 -
142 -     if( particleName == "neutron" )
143 -     { std::ofstream outputFilecompleto("Datos-simulacion/interaccion-completa-neutrones.txt", std:::
144 -         ios_base::app);
145 -     outputFilecompleto << paso << "\t" << particleName << "\t" << trackID << "\t" << parentID << "\t" << E_i /
146 -         MeV << "\t" << x_i << "\t" << y_i << "\t" << z_i << "\t" << x_f << "\t" << y_f << "\t" << z_f << "\t" << E_f / MeV <<
147 -         "\t" << postStepPoint->GetProcessDefinedStep()->GetProcessName() << "\n";
148 -     outputFilecompleto.close();}
149 +G4int CurrentRunId()
150 +{
151 +     const auto* runManager = G4RunManager::GetRunManager();
152 +     const auto* run = runManager ? runManager->GetCurrentRun() : nullptr;
153 +     return run ? run->GetRunID() : -1;
154 +}
155 +
156 +G4int CurrentEventId()
157 +{
158 +     const auto* runManager = G4RunManager::GetRunManager();
159 +     const auto* event = runManager ? runManager->GetCurrentEvent() : nullptr;
160 +     return event ? event->GetEventID() : -1;
161 +}
162 +
163 +// Agrega una linea TSV de forma segura en ejecuciones multihilo. El encabezado
164 +// se escribe una sola vez cuando el archivo no existe o esta vacio.
165 +void AppendTsv(const G4String& fileName,
166 +               const char* header,
167 +               const G4String& row)
168 +{
169 +     G4AutoLock lock(&outputMutex);
170 +
171 +     const G4String path = outputDirectory + fileName;
172 +     std::ifstream input(path.c_str(), std::ios::binary | std::ios::ate);
173 +     const bool needsHeader = !input.good() || input.tellg() == std::streampos(0);
174 +     input.close();
175 +
176 +     std::ofstream output(path.c_str(), std::ios::out | std::ios::app);
177 +     if (!output) {
178 +         G4ExceptionDescription description;
179 +         description << "No se pudo abrir " << path
180 +             << ". Compruebe que el directorio Datos-simulacion existe "
181 +             << "y tiene permisos de escritura.";
182 +         G4Exception("AppendTsv", "WCD-OUTPUT-001", FatalException, description);
183 +         return;
184 +     }
185 +
186 +     if (needsHeader) {
187 +         output << header << '\n';
188 +     }
189 +     output << row << '\n';
190 +}

```

```

184
185 - ////////////////////////////////////////////////////////////////////Hasta aqui//////////////////////////////////////////////////////////////////
186 -
187 - /* if (fReactionsPerEvent.find(processName) == fReactionsPerEvent.end())
188 - {
189 -     // Si el proceso no está registrado, inicializa el conteo
190 -     fReactionsPerEvent[processName] = 1;
191 - }
192 - else {
193 -     // Incrementa el conteo del proceso
194 -     fReactionsPerEvent[processName]++;
195 - }
196 +G4String StepRow(const G4Step* step, const G4String& processName)
197 +{
198 + const auto* track = step->GetTrack();
199 + const auto* pre = step->GetPreStepPoint();
200 + const auto* post = step->GetPostStepPoint();
201 + const auto prePosition = pre->GetPosition();
202 + const auto postPosition = post->GetPosition();
203 +
204 + std::ostringstream row;
205 + row << std::setprecision(10)
206 +     << CurrentRunId() << '\t'
207 +     << CurrentEventId() << '\t'
208 +     << track->GetParticleDefinition()->GetParticleName() << '\t'
209 +     << track->GetTrackID() << '\t'
210 +     << track->GetParentID() << '\t'
211 +     << track->GetCurrentStepNumber() << '\t'
212 +     << ProcessName(track->GetCreatorProcess()) << '\t'
213 +     << processName << '\t'
214 +     << MaterialName(pre) << '\t'
215 +     << prePosition.x() / cm << '\t'
216 +     << prePosition.y() / cm << '\t'
217 +     << prePosition.z() / cm << '\t'
218 +     << postPosition.x() / cm << '\t'
219 +     << postPosition.y() / cm << '\t'
220 +     << postPosition.z() / cm << '\t'
221 +     << pre->GetKineticEnergy() / MeV << '\t'
222 +     << post->GetKineticEnergy() / MeV << '\t'
223 +     << track->GetGlobalTime() / ns;
224 + return row.str();
225 +}
226
227 - // Si es el último paso en el volumen principal, imprime las reacciones del evento
228 - if (step->GetTrack()->GetTrackID() == 1 && step->IsLastStepInVolume())
229 - {
230 -     G4cout << "\n Lista de reacciones nucleares en este evento: \n" << G4endl;
231 -
232 -     for (const auto& reaction : fReactionsPerEvent)
233 -     {
234 -         G4cout << " " << std::setw(30) << reaction.first << ": "
235 -             << std::setw(7) << reaction.second << G4endl;
236 -     }
237 - }*/
238 -
239 -//////////////////////////////////////////////////////////////////para el inicio//////////////////////////////////////////////////////////////////
240 +void WriteSecondaries(const G4Step* step,
241 +                     const G4String& category,
242 +                     const G4String& fileName)
243 +{
244 + const auto* parent = step->GetTrack();
245 + const auto* pre = step->GetPreStepPoint();
246 + const auto* secondaries = step->GetSecondaryInCurrentStep();
247 +
248 + for (std::size_t index = 0; index < secondaries->size(); ++index) {
249 +     const auto* secondary = (*secondaries)[index];
250 +     const auto position = secondary->GetPosition();
251 +
252 +     std::ostringstream row;
253 +     row << std::setprecision(10)
254 +         << CurrentRunId() << '\t'
255 +         << CurrentEventId() << '\t'
256 +         << category << '\t'
257 +         << parent->GetParticleDefinition()->GetParticleName() << '\t'

```

```

258 +     << parent->GetTrackID() << '\t'
259 +     << parent->GetCurrentStepNumber() << '\t'
260 +     << index << '\t'
261 +     << secondary->GetParticleDefinition()->GetParticleName() << '\t'
262 +     << secondary->GetTrackID() << '\t'
263 +     << secondary->GetParentID() << '\t'
264 +     << ProcessName(secondary->GetCreatorProcess()) << '\t'
265 +     << MaterialName(pre) << '\t'
266 +     << position.x() / cm << '\t'
267 +     << position.y() / cm << '\t'
268 +     << position.z() / cm << '\t'
269 +     << secondary->GetKineticEnergy() / MeV << '\t'
270 +     << secondary->GetGlobalTime() / ns;
271
272 + AppendTsv(fileName, secondaryHeader, row.str());
273 + }
274 +}
275 +
276 +void WriteCapture(const G4Step* step, const G4String& processName)
277 +{
278 +     const auto* track = step->GetTrack();
279 +     const auto* pre = step->GetPreStepPoint();
280 +     const auto* post = step->GetPostStepPoint();
281 +     const auto prePosition = pre->GetPosition();
282 +     const auto postPosition = post->GetPosition();
283 +
284 +     G4int targetZ = -1;
285 +     G4int targetA = -1;
286 +     const auto* process = post->GetProcessDefinedStep();
287 +     const auto* hadronicProcess = dynamic_cast<const G4HadronicProcess*>(process);
288 +     if (hadronicProcess && hadronicProcess->GetTargetNucleus()) {
289 +         targetZ = hadronicProcess->GetTargetNucleus()->GetZ_asInt();
290 +         targetA = hadronicProcess->GetTargetNucleus()->GetA_asInt();
291 +     }
292 +
293 +     std::ostringstream row;
294 +     row << std::setprecision(10)
295 +         << CurrentRunId() << '\t'
296 +         << CurrentEventId() << '\t'
297 +         << track->GetParticleDefinition()->GetParticleName() << '\t'
298 +         << track->GetTrackID() << '\t'
299 +         << track->GetParentID() << '\t'
300 +         << track->GetCurrentStepNumber() << '\t'
301 +         << processName << '\t'
302 +         << MaterialName(pre) << '\t'
303 +         << targetZ << '\t'
304 +         << targetA << '\t'
305 +         << prePosition.x() / cm << '\t'
306 +         << prePosition.y() / cm << '\t'
307 +         << prePosition.z() / cm << '\t'
308 +         << postPosition.x() / cm << '\t'
309 +         << postPosition.y() / cm << '\t'
310 +         << postPosition.z() / cm << '\t'
311 +         << pre->GetKineticEnergy() / MeV << '\t'
312 +         << post->GetKineticEnergy() / MeV << '\t'
313 +         << track->GetGlobalTime() / ns;
314
315 + AppendTsv("capturas-neutrones.tsv", captureHeader, row.str());
316 +}
317
318 +bool IsGammaInteraction(const G4String& processName)
319 +{
320 +     return processName == "phot" || processName == "compt" ||
321 +         processName == "conv" || processName == "Rayl";
322 +}
323
324 +bool IsNeutronInteraction(const G4String& processName)
325 +{
326 +     return processName == "hadElastic" || processName == "nCapture" ||
327 +         processName == "neutronInelastic";
328 +}
329
330 - //G4Track* track = step->GetTrack();
331 - //G4String particleName = track->GetDynamicParticle()->GetDefinition()->GetParticleName();

```

```

332 -
333 -
334 - //G4ParticleDefinition* particle = track->GetDefinition();
335 - //G4String motherParticleName = "None";
336 - G4ThreeVector position = track->GetPosition();
337 - G4double x = position.x();
338 - G4double y = position.y();
339 - G4double z = position.z();
340 - //G4double time = track->GetGlobalTime();
341 - G4double energy = track->GetKineticEnergy();
342 - //G4double parentID = step->GetTrack()->GetParentID();
343 - //G4double paso = step->GetTrack()->GetCurrentStepNumber();
344 - //G4int trackID = track->GetTrackID();
345 - // G4String processName = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName();
346 -
347 -
348 -
349 - //G4String processName_pos = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName();
350 - //G4String processNamepre = step->GetPreStepPoint()->GetProcessDefinedStep()->GetProcessName();
351 - G4double preProcessEnergy = step->GetPreStepPoint()->GetKineticEnergy();
352 - G4double postProcessEnergy = step->GetPostStepPoint()->GetKineticEnergy();
353 -
354 -
355 -//if (parentID > 0) {
356 - //G4String motherParticleName = track->GetDynamicParticle()->GetPrimaryParticle()->
357 - //    GetParticleDefinition()->GetParticleName();
358 - // }
359 -
360 - std::ofstream outputFileparticles("Datos-simulacion/rastreo-completo-particulas.txt", std::ios_base::
361 - app);
362 - outputFileparticles << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
363 - processName << "\t" << x << "\t" << y << "\t" << z << "\t" << preProcessEnergy << "\t" << energy / MeV <<
364 - "\t" << postProcessEnergy << "\n";
365 - outputFileparticles.close();
366 -
367 - //if (particleName == "neutron" )
368 - // {
369 -
370 - // G4double zTopCap = 66.5*cm; // Cambia si la tapa está en otra altura
371 - // G4double radiusCylinder = 48.0 *cm; // Radio del cilindro
372 -
373 - //if (z == zTopCap && std::sqrt(x*x + y*y) <= radiusCylinder)
374 - // {
375 -
376 - //std::ofstream outputFileneutroncruce("Datos-simulacion/neutrons_crossing_top.txt", std::
377 - //ios_base::app);
378 - //outputFileneutroncruce << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso
379 - //<< "\t" << x << "\t" << y << "\t" << z << "\t" << preProcessEnergy << "\t" << energy / MeV << "\t" <<
380 - //postProcessEnergy << "\n";
381 - //}
382 - // }
383 -
384 - // Log basic neutron data
385 -
386 - if (particleName == "neutron")
387 - {
388 -
389 - std::ofstream outputFileneutron("Datos-simulacion/rastreo-neutron-completo.txt", std::ios_base::app);
390 - outputFileneutron << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
391 - processName << "\t" << x << "\t" << y << "\t" << z << "\t" << preProcessEnergy << "\t" << energy / MeV
392 - << "\t" << postProcessEnergy << "\n";
393 -
394 - //outputFileneutron.close();
395 -
396 -
397 - //std::ofstream outputFile("captured_neutrons.txt", std::ios_base::app);
398 - // output << "Energy: " << energy / CLHEP::eV << " eV, "
399 - // << "Position: " << position.x() / CLHEP::cm << " cm, "
400 - // << position.y() / CLHEP::cm << " cm, "
401 - // << position.z() / CLHEP::cm << " cm\n";

```

```

397 - // outputFile.close();
398 -
399 -
400 -
401 - //G4double zTopCap = 130*cm; // Cambia si la tapa está en otra altura
402 - //G4double radiusCylinder = 48.0 *cm; // Radio del cilindro
403 -
404 - //if (position.z() <= zTopCap && std::sqrt(position.x() * position.x() + position.y() * position.y())
405 - <= radiusCylinder)
406 - // {
407 - //std::ofstream outputFileneutroncruce("Datos-simulacion/neutrons_crossing_top.txt", std::ios_base
408 - ::app);
409 - //outputFileneutroncruce << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t"
410 - << x << "\t" << y << "\t" << z << "\t" << preProcessEnergy << "\t" << energy / MeV << "\t" <<
411 - postProcessEnergy << "\n";
412 - //}
413 -
414 - if (processName == "nCapture" ) //parentID == 0
415 - {
416 - //const G4Nucleus* targetNucleus = hadronicProcess->GetTargetNucleus();
417 - //if (!targetNucleus) return;
418 -
419 - // Identificar si el núcleo objetivo es hidrógeno
420 - //G4int Z = targetNucleus->GetZ_asInt();
421 - //G4int A = targetNucleus->GetA_asInt();
422 -
423 - //G4String materialName = step->GetPreStepPoint()->GetMaterial()->GetName();
424 - //if (materialName != "Water") return;
425 - //if (parentID != 1 ) return;
426 -
427 - G4ThreeVector captureposition = track->GetPosition();
428 - G4double xcn = captureposition.x();
429 - G4double ycn = captureposition.y();
430 - G4double zcn = captureposition.z();
431 - //G4double captureEnergy = track->GetKineticEnergy();
432 - G4double preCaptureEnergy = step->GetPreStepPoint()->GetKineticEnergy();
433 - G4double postCaptureEnergy = step->GetPostStepPoint()->GetKineticEnergy();
434 - //G4String processNamepos = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName();
435 - //G4String processNamepre = step->GetPreStepPoint()->GetProcessDefinedStep()->GetProcessName();
436 -
437 - // Identificar el elemento del núcleo objetivo
438 - const G4Material* material = step->GetPreStepPoint()->GetMaterial();
439 - const G4Element* elemento = material->GetElement(0); // Primer elemento del material
440 - G4String elementName = elemento->GetName();
441 - G4int Z = elemento->GetZ(); // Número atómico
442 - // G4int A = elemento->GetN(); // Número de masa si está disponible
443 -
444 - // Abrir archivos para registrar los datos
445 - std::ofstream outputFileTotal("Datos-simulacion/neutrones-capturados.txt", std::ios_base::app);
446 -
447 - if (Z == 8 && elementName == "Oxygen") { // Oxigeno
448 - std::ofstream outputFileCl35("Datos-simulacion/neutrones-capturados-Cl35.txt", std::ios_base::app
449 - );
450 - outputFileCl35 << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
451 - processName << "\t" << xcn << "\t" << ycn << "\t" << zcn << "\t" << preCaptureEnergy << "\t" << energy
452 - / MeV << "\t" << postCaptureEnergy << "\n";
453 - outputFileCl35.close();
454 - } else if (Z == 1 &&
455 - (elementName == "Hydrogen" ||
456 - elementName == "TS_H_of_Water")) { // Hidrógeno libre o ligado
457 - std::ofstream outputFileH("Datos-simulacion/neutrones-capturados-hidrogeno.txt", std::ios_base::
458 - app);
459 - outputFileH << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
460 - processName << "\t" << xcn << "\t" << ycn << "\t" << zcn << "\t" << preCaptureEnergy << "\t" << energy
461 - / MeV << "\t" << postCaptureEnergy << "\n";
462 - outputFileH.close();
463 - }
464 - void WriteCerenkovPhotons(const G4Step* step)
465 - {
466 - const auto* parent = step->GetTrack();
467 - const auto* pre = step->GetPreStepPoint();
468 - const auto* secondaries = step->GetSecondaryInCurrentStep();

```

```

461 +
462 + for (std::size_t index = 0; index < secondaries->size(); ++index) {
463 +     const auto* secondary = (*secondaries)[index];
464 +     const auto* creator = secondary->GetCreatorProcess();
465 +
466 +     if (secondary->GetParticleDefinition() != G4OpticalPhoton::Definition() ||
467 +         ProcessName(creator) != "Cerenkov") {
468 +         continue;
469 +     }
470 +
471 +     // Registrar en el archivo general
472 +     outputFileTotal << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
processName << "\t" << xcn << "\t" << ycn << "\t" << zcn << "\t" << preCaptureEnergy << "\t" << energy
/ MeV << "\t" << postCaptureEnergy << "\n";
473 +     outputFileTotal.close();
474 +
475 +     // std::ofstream outputFileCerenkov1("Datos-simulacion/neutrones-capturados.txt", std::ios_base::
app);
476 +     // outputFileCerenkov1 << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t"
<< processName << "\t" << xcn << "\t" << ycn << "\t" << zcn << "\t" << preCaptureEnergy << "\t" <<
energy / MeV << "\t" << postCaptureEnergy << "\n";
477 +
478 +     //outputFileCerenkov1.close();
479 +     // Get secondary particles of capture
480 +     const std::vector<const G4Track*> secondaries = step->GetSecondaryInCurrentStep();
481 +     for (size_t i = 0; i < secondaries->size(); ++i)
482 +     {
483 +         const G4Track* secondary = (*secondaries)[i];
484 +         G4String particleNamesp = secondary->GetDefinition()->GetParticleName();
485 +         G4ThreeVector positionsp = secondary->GetPosition();
486 +         G4double xsp = positionsp.x();
487 +         G4double ysp = positionsp.y();
488 +         G4double zsp = positionsp.z();
489 +         G4double energysp = secondary->GetKineticEnergy();
490 +         G4double parentIDspCn = step->GetTrack()->GetParentID();
491 +         G4double pasospcn = step->GetTrack()->GetCurrentStepNumber();
492 +         G4int trackIDspCn = track->GetTrackID();
493 +         std::ofstream outputFileCerenkovsc("Datos-simulacion/secondary-particles-neutron-capturado.
txt", std::ios_base::app);
494 +         outputFileCerenkovsc << particleNamesp << "\t" << trackIDspCn << "\t" << parentIDspCn << "\t"
<< pasospcn << "\t" << processName << "\t" << xsp << "\t" << ysp << "\t" << zsp << "\t" << energysp /
MeV << "\n";
495 +
496 +         outputFileCerenkovsc.close();
497 +
498 +     }
499 +     // outputFileCerenkov1.close();
500 + }
501 +
502 +
503 + if (processName == "neutronInelastic" )
504 + {
505 +     G4ThreeVector ninelasticposition = track->GetPosition();
506 +     G4double xni = ninelasticposition.x();
507 +     G4double yni = ninelasticposition.y();
508 +     G4double zni = ninelasticposition.z();
509 +     G4double ninelasticEnergy = track->GetKineticEnergy();
510 +
511 +     std::ofstream outputFileCerenkovin("Datos-simulacion/neutron-inelastico-punto.txt", std:::
ios_base::app);
512 +     outputFileCerenkovin << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t"
<< processName << "\t" << xni << "\t" << yni << "\t" << zni << "\t" << ninelasticEnergy / MeV << "\n";
513 +
514 +     outputFileCerenkovin.close();
515 +     // Get secondary particles
516 +     const std::vector<const G4Track*> secondariesni = step->GetSecondaryInCurrentStep();
517 +     for (size_t j = 0; j < secondariesni->size(); ++j)
518 +     {
519 +         const G4Track* secondaryni = (*secondariesni)[j];
520 +         G4String particleNamesni = secondaryni->GetDefinition()->GetParticleName();
521 +         G4ThreeVector positionni = secondaryni->GetPosition();
522 +         G4double xin = positionni.x();
523 +         G4double yin = positionni.y();
524 +         G4double zin = positionni.z();

```

```

525 -         G4double energyni = secondaryni->GetKineticEnergy();
526 -
527 -         std::ofstream outputFilecaneutrons("Datos-simulacion/secondary-particles-neutron-
inelastic.txt", std::ios_base::app);
528 -         outputFilecaneutrons << particleNamesni << "\t" << trackID << "\t" << parentID << "\t" <<
paso << "\t" << processName << "\t" << xin << "\t" << yin << "\t" << zin << "\t" << energyni / MeV << "\n
";
529 -         outputFilecaneutrons.close();
530 -     }
531 - }
532 -     outputFileneutron.close();
533 - }
534 - }
535 - //}
536 - ///////////////////////////////////SECCION GAMMAS
537 - ///////////////////////////////////
538 - //if (particleName == "gamma" && parentID == 1) {
539 - if (particleName == "gamma")
540 - {
541 -     std::ofstream outputFilegamma1("Datos-simulacion/gamma-completo.txt", std::ios_base::app);
542 -     outputFilegamma1 << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t" <<
processName << "\t" << x << "\t" << y << "\t" << z << "\t" << energy / MeV << "\n";
543 -     outputFilegamma1.close();
544 -
545 -     if (processName == "phot" || processName == "compt" || processName == "conv" || processName=="Rayl"
)
546 -     {
547 -         /// processName=="Rayl"
548 -         G4ThreeVector comptposition = track->GetPosition();
549 -         G4double xcmt = comptposition.x();
550 -         G4double ycmt = comptposition.y();
551 -         G4double zcmt = comptposition.z();
552 -         G4double comptEnergy = track->GetKineticEnergy();
553 -         G4double precomptEnergy = step->GetPreStepPoint()->GetKineticEnergy();
554 -         G4double postcomptEnergy = step->GetPostStepPoint()->GetKineticEnergy();
555 -         //G4String posprocessName = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName
556 -         ();
557 -         //G4String preprocessName = step->GetPreStepPoint()->GetProcessDefinedStep()->GetProcessName()
558 -         ;
559 -         std::ofstream outputFilegamma2compt("Datos-simulacion/gamma-compt-punto.txt", std::ios_base::app);
560 -         outputFilegamma2compt << particleName << "\t" << trackID << "\t" << parentID << "\t" << paso << "\t"
<< processName << "\t" << xcmt << "\t" << ycmt << "\t" << zcmt << "\t" << precomptEnergy << "\t"
<< comptEnergy / MeV << "\t" << postcomptEnergy << "\n";
561 -         outputFilegamma2compt.close();
562 -
563 -         // Get secondary particles
564 -         const std::vector<const G4Track*> secondariescompt = step->GetSecondaryInCurrentStep();
565 -         for (size_t k = 0; k < secondariescompt->size(); ++k)
566 -         {
567 -             const G4Track* secondarycompt = (*secondariescompt)[k];
568 -             G4String particleNamescompt = secondarycompt->GetDefinition()->GetParticleName();
569 -             G4ThreeVector positioncompt = secondarycompt->GetPosition();
570 -             G4double xpcompt = positioncompt.x();
571 -             G4double ypcompt = positioncompt.y();
572 -             G4double zpcompt = positioncompt.z();
573 -             G4double energycompt = secondarycompt->GetKineticEnergy();
574 -             G4double Energyprecompt = step->GetPreStepPoint()->GetKineticEnergy();
575 -             G4double Energypostcompt = step->GetPostStepPoint()->GetKineticEnergy();
576 -
577 -             std::ofstream outputFilesecparcompt("Datos-simulacion/secondary-particles-gamma-compton.
txt", std::ios_base::app);
578 -             outputFilesecparcompt << particleNamescompt << "\t" << trackID << "\t" << parentID << "\t" <<
paso << "\t" << processName << "\t" << xpcompt << "\t" << ypcompt << "\t" << zpcompt << "\t" <<
Energyprecompt << "\t" << energycompt / MeV << "\t" << Energypostcompt << "\n";
579 -             outputFilesecparcompt.close();
580 -
581 -         }
582 -     }
583 - }
584 - }
585 - ///////////////////////////////////SECCION ELECTRONES
586 - ///////////////////////////////////
587 - if (particleName == "e-" || particleName == "e+")
588 - {

```

```

585 -         std::ofstream outputFileElectron("Datos-simulacion/rastreo-electron.txt", std::ios_base::app);
586 -         outputFileElectron << particleName << "\t" << processName << "\t" << trackID << "\t" << parentID <<
"\t" << paso << "\t" << x << "\t" << y << "\t" << z << "\t" << energy / MeV << "\n";
587 -
588 -     outputFileElectron.close();
589 -     if (processName == "Cerenkov" )
590 -     {
591 -         G4ThreeVector cerenkovposition = track->GetPosition();
592 -         G4double xck = cerenkovposition.x();
593 -         G4double yck = cerenkovposition.y();
594 -         G4double zck = cerenkovposition.z();
595 -         //G4double captureEnergy = track->GetKineticEnergy();
596 -         //G4double preCerenkovEnergy = step->GetPreStepPoint()->GetKineticEnergy();
597 -         //G4double postCerenkovEnergy = step->GetPostStepPoint()->GetKineticEnergy();
598 -         //G4String processNamepos = step->GetPostStepPoint()->GetProcessDefinedStep()->GetProcessName
();
599 -         //G4String processNamepre = step->GetPreStepPoint()->GetProcessDefinedStep()->GetProcessName
();
600 -
601 -
602 -         std::ofstream outputFileCerenkov("Datos-simulacion/radiacion-Cerenkov-electron.txt", std::ios_base::
app);
603 -         outputFileCerenkov << particleName << "\t" << processName << "\t" << trackID << "\t" << parentID
<< "\t" << paso << "\t" << xck << "\t" << yck << "\t" << zck << "\t" << energy / MeV << "\n";
604 -         outputFileCerenkov.close();
605 -     }
606 - }
607 - //outputFileCaneutrons.close();
608 - //outputFileCaneutron1.close();
609 - //outputFileCaneutronin.close();
610 - //outputFileCaneutronsin.close();
611 - //outputFilegamma1.close();
612 - //outputFilegammacompt.close();
613 - //outputFileseccparcompt.close();
614 - //outputFileelectron.close();
615 - //outputFileCerenkov.close();
616 + const auto position = secondary->GetPosition();
617 + std::ostringstream row;
618 + row << std::setprecision(10)
619 + << CurrentRunId() << '\t'
620 + << CurrentEventId() << '\t'
621 + << "Cerenkov" << '\t'
622 + << parent->GetParticleDefinition()->GetParticleName() << '\t'
623 + << parent->GetTrackID() << '\t'
624 + << parent->GetCurrentStepNumber() << '\t'
625 + << index << '\t'
626 + << secondary->GetParticleDefinition()->GetParticleName() << '\t'
627 + << secondary->GetTrackID() << '\t'
628 + << secondary->GetParentID() << '\t'
629 + << ProcessName(creator) << '\t'
630 + << MaterialName(pre) << '\t'
631 + << position.x() / cm << '\t'
632 + << position.y() / cm << '\t'
633 + << position.z() / cm << '\t'
634 + << secondary->GetKineticEnergy() / MeV << '\t'
635 + << secondary->GetGlobalTime() / ns;
636 +
637 + AppendTsv("fotones-cherenkov.tsv", secondaryHeader, row.str());
638 + }
639 +}
640 +} // namespace
641 +
642 +G4WCSteppingAction::G4WCSteppingAction(
643 + const G4WCConstruction* detector,
644 + G4WCEventAction* eventAction,
645 + Event& event)
646 + : G4UserSteppingAction(),
647 + fDetectorConstruction(detector),
648 + fEventAction(eventAction),
649 + fEvent(event)
650 + {}
651 +
652 +void G4WCSteppingAction::UserSteppingAction(const G4Step* step)
653 +{

```

```

654 + if (!step || !step->GetTrack() || !step->GetPreStepPoint() ||
655 +     !step->GetPostStepPoint()) {
656 +     return;
657 + }
658 +
659 + const auto* track = step->GetTrack();
660 + const G4String particleName =
661 +     track->GetParticleDefinition()->GetParticleName();
662 + const G4String processName =
663 +     ProcessName(step->GetPostStepPoint()->GetProcessDefinedStep());
664 + const G4String row = StepRow(step, processName);
665 +
666 + // Historia completa. Puede generar archivos grandes; comentese esta linea
667 + // cuando solo se necesiten los productos filtrados.
668 + AppendTsv("pasos-particulas.tsv", stepHeader, row);
669 +
670 + if (particleName == "neutron") {
671 +     AppendTsv("pasos-neutrones.tsv", stepHeader, row);
672 +
673 +     // Solo los neutrones primarios incidentes, no neutrones secundarios.
674 +     if (track->GetParentID() == 0 && track->GetCurrentStepNumber() == 1) {
675 +         AppendTsv("neutrones-incidentes.tsv", stepHeader, row);
676 +     }
677 +
678 +     if (IsNeutronInteraction(processName)) {
679 +         AppendTsv("interacciones-neutrones.tsv", stepHeader, row);
680 +     }
681 +
682 +     if (processName == "nCapture") {
683 +         WriteCapture(step, processName);
684 +         WriteSecondaries(step,
685 +             "nCapture",
686 +             "secundarias-captura-neutron.tsv");
687 +     } else if (processName == "neutronInelastic") {
688 +         WriteSecondaries(step,
689 +             "neutronInelastic",
690 +             "secundarias-inelastica-neutron.tsv");
691 +     }
692 + }
693 +
694 + fReactionsPerEvent.clear();
695 + if (particleName == "gamma") {
696 +     AppendTsv("pasos-gamma.tsv", stepHeader, row);
697 +     if (IsGammaInteraction(processName)) {
698 +         AppendTsv("interacciones-gamma.tsv", stepHeader, row);
699 +         WriteSecondaries(step,
700 +             processName,
701 +             "secundarias-interaccion-gamma.tsv");
702 +     }
703 + }
704 +
705 + if (particleName == "e-" || particleName == "e+") {
706 +     AppendTsv("pasos-electrones.tsv", stepHeader, row);
707 + }
708 +
709 + // Cerenkov se reconoce por el proceso creador del foton optico. No suele
710 + // ser el proceso que limita el paso de la particula cargada.
711 + WriteCerenkovPhotons(step);
712 + }

```

A.3. G4Models/Materials.h

```

1 --- MEIGA-anterior/src/G4Models/Materials.h
2 +++ MEIGA-integrado/src/G4Models/Materials.h
3 @@ -62,6 +62,11 @@
4     static G4Material* HDPE;
5     static G4Material* Concrete;
6     static G4Material* SaltyWater;
7 + // Discrete saline media at fixed final NaCl mass fractions (2.5, 5, 10 %).
8 + // Selected by SaltyWCD::BuildDetector according to Detector::GetImpuritiesFraction().
9 + static G4Material* SaltyWater_2p5;
10 + static G4Material* SaltyWater_5;
11 + static G4Material* SaltyWater_10;

```

```

12 static G4OpticalSurface* ScinOptSurf;
13 static G4OpticalSurface* LinerOptSurf;
14 static G4OpticalSurface* LinerOptSurf2;

```

A.4. G4Models/Materials.cc

```

1  --- MEIGA-anterior/src/G4Models/Materials.cc
2  +++ MEIGA-integrado/src/G4Models/Materials.cc
3  @@ -1,5 +1,10 @@
4  #include "Materials.h"
5  #include "G4NistManager.hh"
6  #include "G4Exception.hh"
7  #include <sstream>
8  #include <iomanip>
9  #include <algorithm>
10 #include <cmath>
11 // material properties (scintillation, Cherenkov, optics)
12
13 // array of photon energies in the red-UV range for scintillation processes
14 @@ -116,7 +121,11 @@
15 1.42, 1.42, 1.42, 1.42, 1.42, 1.42, 1.42, 1.42, 1.42, 1.42
16 };
17
18 -static G4double pmtPhotonEnergy[] = {2.00*eV, 4.00*eV};
19 // Effective PMT absorber: cover the same 300--650 nm response range used by
20 // OptDevice::GetQuantumEfficiency. RINDEX controls the Fresnel probability at
21 // the active-medium/PMT boundary; the very short ABSLENGTH makes transmission
22 // into the solid terminal. QE is applied once in G4MPMTAction, not here.
23 +static G4double pmtPhotonEnergy[] = {(1240.0/650.0)*eV, (1240.0/300.0)*eV};
24 static G4double pmtRefIndex[] = {1.47, 1.47};
25 static G4double pmtAbsLength[] = {0.0005*mm, 0.0005*mm};
26 G4int pmtArrEntries = sizeof(pmtPhotonEnergy) / sizeof(G4double);
27 @@ -161,22 +170,57 @@
28 4.002 * eV, 4.136 * eV
29 };
30
31 -static G4double water2RefIndex[] = {
32 - 1.3435, 1.344, 1.3445, 1.345, 1.3455, 1.346, 1.3465, 1.347,
33 - 1.3475, 1.348, 1.3485, 1.3492, 1.35, 1.3505, 1.351, 1.3518,
34 - 1.3522, 1.3530, 1.3535, 1.354, 1.3545, 1.355, 1.3555, 1.356,
35 - 1.3568, 1.3572, 1.358, 1.3585, 1.359, 1.3595, 1.36, 1.3608
36 - };
37 -
38 -G4double scaleAbsLen2 = 1.0*m;
39 -static G4double water2AbsLen[] = {
40 - 3.448*scaleAbsLen2, 4.082*scaleAbsLen2, 6.329*scaleAbsLen2, 9.174*scaleAbsLen2, 12.346*scaleAbsLen2,
41 - 13.889*scaleAbsLen2,
42 - 15.152*scaleAbsLen2, 17.241*scaleAbsLen2, 18.868*scaleAbsLen2, 20.000*scaleAbsLen2, 26.316*scaleAbsLen2,
43 - 35.714*scaleAbsLen2,
44 - 45.455*scaleAbsLen2, 47.619*scaleAbsLen2, 52.632*scaleAbsLen2, 52.632*scaleAbsLen2, 55.556*scaleAbsLen2,
45 - 52.632*scaleAbsLen2, 47.619*scaleAbsLen2, 45.455*scaleAbsLen2, 41.667*scaleAbsLen2, 37.037*scaleAbsLen2,
46 - 33.333*scaleAbsLen2,
47 - 30.000*scaleAbsLen2, 28.500*scaleAbsLen2, 27.000*scaleAbsLen2, 24.500*scaleAbsLen2, 22.000*scaleAbsLen2,
48 - 19.500*scaleAbsLen2,
49 - 17.500*scaleAbsLen2, 14.500*scaleAbsLen2
50 - };
51 // WCD refractive-index spectra on the same 32-point energy grid above.
52 // Arrays are intentionally non-const for compatibility with the
53 // G4MaterialPropertiesTable::AddProperty API in Geant4 10.07.p04.
54 +static G4double pureWaterRefIndex[] = {
55 + 1.3319872, 1.3324872, 1.3329872, 1.3334872, 1.3339872, 1.3344872,
56 + 1.3349872, 1.3354872, 1.3359872, 1.3364872, 1.3369872, 1.3376872,
57 + 1.3384872, 1.3389872, 1.3394872, 1.3402872, 1.3406872, 1.3414872,
58 + 1.3419872, 1.3424872, 1.3429872, 1.3434872, 1.3439872, 1.3444872,
59 + 1.3452872, 1.3456872, 1.3464872, 1.3469872, 1.3474872, 1.3479872,
60 + 1.3484872, 1.3492872
61 +};
62 +
63 +static G4double saltyWater2p5RefIndex[] = {
64 + 1.3389748, 1.3393319, 1.3396866, 1.3400397, 1.3403883, 1.3407376,
65 + 1.3410888, 1.3414396, 1.3417951, 1.3421537, 1.3425175, 1.3430903,
66 + 1.3437697, 1.3441624, 1.3445696, 1.3452923, 1.3456353, 1.3463984,

```

```

63 + 1.3468882, 1.3474063, 1.3479562, 1.3485422, 1.3491666, 1.3498363,
64 + 1.3508518, 1.3515223, 1.3526517, 1.3535431, 1.3545063, 1.3555405,
65 + 1.3566576, 1.3581655
66 +};
67 +
68 +static G4double saltyWater5RefIndex[] = {
69 + 1.3428145, 1.3431976, 1.3435767, 1.3439527, 1.3443232, 1.3446927,
70 + 1.3450624, 1.3454307, 1.3458022, 1.3461759, 1.3465541, 1.3471392,
71 + 1.3478302, 1.3482365, 1.3486578, 1.3493938, 1.3497542, 1.3505348,
72 + 1.3510465, 1.3515907, 1.3521714, 1.3527942, 1.3534617, 1.3541829,
73 + 1.3552573, 1.3559996, 1.3572114, 1.3581999, 1.3592765, 1.3604404,
74 + 1.3617071, 1.3633853
75 +};
76 +
77 +static G4double saltyWater10RefIndex[] = {
78 + 1.3588310, 1.3590937, 1.3593505, 1.3596026, 1.3598447, 1.3600849,
79 + 1.3603250, 1.3605609, 1.3608010, 1.3610423, 1.3612880, 1.3617427,
80 + 1.3623022, 1.3625837, 1.3628854, 1.3635060, 1.3637615, 1.3644425,
81 + 1.3648698, 1.3653417, 1.3658659, 1.3664496, 1.3670986, 1.3678241,
82 + 1.3689258, 1.3697281, 1.3710292, 1.3721451, 1.3733899, 1.3747644,
83 + 1.3762907, 1.3782804
84 +};
85 +
86 +// Common absorption-length spectrum. In the absence of concentration-specific
87 +// measurements, the same baseline is used for all four WCD media.
88 +static G4double waterAbsLen[] = {
89 + 3.448*m, 4.082*m, 6.329*m, 9.174*m,
90 + 12.346*m, 13.889*m, 15.152*m, 17.241*m,
91 + 18.868*m, 20.000*m, 26.316*m, 35.714*m,
92 + 45.455*m, 47.619*m, 52.632*m, 52.632*m,
93 + 55.556*m, 52.632*m, 52.632*m, 47.619*m,
94 + 45.455*m, 41.667*m, 37.037*m, 33.333*m,
95 + 30.000*m, 28.500*m, 27.000*m, 24.500*m,
96 + 22.000*m, 19.500*m, 17.500*m, 14.500*m
97 +};
98 +
99 +
100 +static G4double water2PhotonEnergyMie[] = {
101 @@ -217,32 +261,57 @@
102 // gforward, gbackward, forward backward ratio
103 static G4double water2MieConst[3] = { 0.99, 0.99, 0.8 };
104
105 +static G4MaterialPropertiesTable* CreateWCDOpticalTable(
106 + G4double* refractiveIndex)
107 +{
108 + G4MaterialPropertiesTable* table = new G4MaterialPropertiesTable();
109 + table->AddProperty("RINDEX",
110 + water2PhotonEnergy,
111 + refractiveIndex,
112 + water2ArrEntries);
113 + table->AddProperty("ABSLLENGTH",
114 + water2PhotonEnergy,
115 + waterAbsLen,
116 + water2ArrEntries);
117 +
118 + // Retain the pre-existing Geant4 OpNovice Mie model as a common baseline.
119 + // It is not a measured salinity-dependent scattering model.
120 + table->AddProperty("MIEHG",
121 + water2PhotonEnergyMie,
122 + water2Mie,
123 + sizeof(water2PhotonEnergyMie)/sizeof(G4double));
124 + table->AddConstProperty("MIEHG_FORWARD", water2MieConst[0]);
125 + table->AddConstProperty("MIEHG_BACKWARD", water2MieConst[1]);
126 + table->AddConstProperty("MIEHG_FORWARD_RATIO", water2MieConst[2]);
127 + return table;
128 +}
129 +
130 +
131 +
132 // ----- Liner (Tyvek) -----
133
134 static G4double linerAbsLen[] = {
135 - 10*m, 10*m, 10*m, 10*m, 10*m, 10*m,
136 - 10*m, 10*m, 10*m, 10*m, 10*m, 10*m,

```

```

137 - 10*mm, 10*mm, 10*mm, 10*mm, 10*mm, 10*mm,
138 - 10*mm, 10*mm, 10*mm, 10*mm, 10*mm, 10*mm,
139 - 10*mm, 10*mm, 10*mm, 10*mm, 10*mm, 10*mm
140 + 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm,
141 + 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm,
142 + 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm,
143 + 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm,
144 + 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm, 0.1*mm
145 };
146
147 static G4double linerRefIndex[] = {
148 - 0, 0, 0, 0, 0, 0,
149 - 0, 0, 0, 0, 0, 0,
150 - 0, 0, 0, 0, 0, 0,
151 - 0, 0, 0, 0, 0, 0,
152 - 0, 0, 0, 0, 0, 0
153 + 1.49, 1.49, 1.49, 1.49, 1.49, 1.49,
154 + 1.49, 1.49, 1.49, 1.49, 1.49, 1.49,
155 + 1.49, 1.49, 1.49, 1.49, 1.49, 1.49,
156 + 1.49, 1.49, 1.49, 1.49, 1.49, 1.49,
157 + 1.49, 1.49, 1.49, 1.49, 1.49, 1.49
158 };
159
160 static G4double linerBackScatter[] = {
161 - 0, 0, 0, 0, 0, 0,
162 - 0, 0, 0, 0, 0, 0,
163 - 0, 0, 0, 0, 0, 0,
164 - 0, 0, 0, 0, 0, 0,
165 - 0, 0, 0, 0, 0, 0
166 + 0.02, 0.02, 0.02, 0.02, 0.02, 0.02,
167 + 0.02, 0.02, 0.02, 0.02, 0.02, 0.02,
168 + 0.02, 0.02, 0.02, 0.02, 0.02, 0.02,
169 + 0.02, 0.02, 0.02, 0.02, 0.02, 0.02,
170 + 0.02, 0.02, 0.02, 0.02, 0.02, 0.02
171 };
172
173 static G4double scaleLinerRef = 0.94;
174 @@ -258,9 +327,100 @@
175 static G4double fSigmaAlpha = 0.17; // 0.23 in doi = 10.1364/OE.19.004199
176 static G4double specularLobePhotonEnergy[] = {2.08*eV, 3.0*eV, 4.20*eV};
177 // fraction of light reflected in a lobe with width characterized by sigmaAlpha
178 -static G4double specularLobe[] = {0.2, 0.2, 0.2}; // same value in doi = 10.1364/OE.19.004199
179 +static G4double specularLobe[] = {0.02, 0.02, 0.02}; // same value in doi = 10.1364/OE.19.004199
180 static G4double specularSpike[] = {0, 0, 0};
181
182 +// -----
183 +// Utilities for saline water at 20 C
184 +// -----
185 +static G4double NormalizeNaClFraction(G4double x)
186 +{
187 + // r_NaCl = m_NaCl / (m_H2O + m_NaCl).
188 + // Accept both 0.025 and 2.5 as input representations of 2.5% w/w.
189 + if (x > 1.0) x /= 100.0;
190 + return x;
191 +}
192 +
193 +static G4double WCDMediumDensity20C(G4double wNaCl)
194 +{
195 + // Tabulated densities used by this study at 20 degC.
196 + // No interpolation is performed: only the four simulated media are valid.
197 + const G4double tol = 1.0e-9;
198 + if (std::abs(wNaCl - 0.000) < tol) return 0.99820 * g/cm3;
199 + if (std::abs(wNaCl - 0.025) < tol) return 1.01604 * g/cm3;
200 + if (std::abs(wNaCl - 0.050) < tol) return 1.03401 * g/cm3;
201 + if (std::abs(wNaCl - 0.100) < tol) return 1.07051 * g/cm3;
202 +
203 + G4ExceptionDescription description;
204 + description << "Unsupported NaCl final mass fraction: " << wNaCl
205 + << ". Valid values are 0, 0.025, 0.050 and 0.100.";
206 + G4Exception("WCDMediumDensity20C", "WCD-MAT-001",
207 + FatalException, description);
208 + return 0.0;
209 +}
210 +

```

```

211 +static G4String SaltyWaterName(G4double wNaCl)
212 +{
213 + const G4double w = NormalizeNaClFraction(wNaCl);
214 + std::ostringstream os;
215 + os << "SaltyWater_NaCl_"
216 + << std::fixed << std::setprecision(1)
217 + << 100.0*w << "pct";
218 + return os.str();
219 +}
220 +
221 +static G4Material* CreateSaltyWaterByFinalMassFraction(
222 + const G4String& name,
223 + G4double wNaClInput,
224 + G4Element* elBoundH,
225 + G4Element* elO,
226 + G4Element* elNa,
227 + G4Element* elCl,
228 + G4MaterialPropertiesTable* opticalMPT
229 +)
230 +{
231 + // Final solution mass fraction:
232 + //  $w_{NaCl} = m_{NaCl} / (m_{H2O} + m_{NaCl})$ .
233 + const G4double wNaCl = NormalizeNaClFraction(wNaClInput);
234 +
235 + const G4double MH = 1.00794;
236 + const G4double MO = 15.9994;
237 + const G4double MNa = 22.98977;
238 + const G4double MCl = 35.453;
239 +
240 + const G4double MH2O = 2.0*MH + MO;
241 + const G4double MNaCl = MNa + MCl;
242 +
243 + const G4double wWater = 1.0 - wNaCl;
244 +
245 + const G4double wH = wWater * (2.0*MH/MH2O);
246 + const G4double wO = wWater * (MO/MH2O);
247 + const G4double wNa = wNaCl * (MNa/MNaCl);
248 + const G4double wCl = wNaCl * (MCl/MNaCl);
249 +
250 + const G4double density = WCDMediumDensity20C(wNaCl);
251 +
252 + // Same thermal state as Water_TS_H_of_Water, and the same bound-hydrogen
253 + // element, so that every WCD saline medium also receives the h_water
254 + // S(alpha,beta) treatment below 4 eV (see G4MPhysicsList::G4MPhysicsList).
255 + // Na, Cl and O keep the standard G4NDL free-gas treatment: G4NDL 4.6 has
256 + // no concentration-specific thermal scattering law for these solutions.
257 + G4Material* saltyWater = new G4Material(name, density, 4,
258 +                                     kStateLiquid,
259 +                                     293.15 * kelvin,
260 +                                     1.0 * bar);
261 + saltyWater->AddElement(elBoundH, wH);
262 + saltyWater->AddElement(elO, wO);
263 + saltyWater->AddElement(elNa, wNa);
264 + saltyWater->AddElement(elCl, wCl);
265 +
266 + // Each saline medium receives its own concentration-dependent optical
267 + // table (see CreateWCDOpticalTable / pureWaterRefIndex / saltyWaterNRefIndex).
268 + if (opticalMPT) saltyWater->SetMaterialPropertiesTable(opticalMPT);
269 +
270 + return saltyWater;
271 +}
272 +
273 +
274 +/////
275 +static bool isInitialized = 0;
276 +@@ -292,6 +452,9 @@
277 + G4Material* Materials::Water;
278 + G4Material* Materials::Salt;
279 + G4Material* Materials::SaltyWater;
280 + G4Material* Materials::SaltyWater_2p5;
281 + G4Material* Materials::SaltyWater_5;
282 + G4Material* Materials::SaltyWater_10;
283 + G4Material* Materials::StdRock;
284 + G4Material* Materials::ScinPlastic;

```

```

285 G4Material* Materials::ScinCoating;
286 @@ -354,6 +517,8 @@
287   elH = new G4Element("Hydrogen", "H", 1, 1.01 * g/mole);
288   // Exact Geant4 thermal-scattering name. G4NeutronHP maps this
289   // pseudo-element to the h_water S(alpha,beta) evaluation in G4NDL.
290 + // Used for every WCD active medium (pure water and the three NaCl
291 + // solutions) so hydrogen is consistently treated as bound, not free-gas.
292   elTSHWater = new G4Element("TS_H_of_Water", "H_WATER",
293                               1.0, 1.00794 * g/mole);
294   elC = new G4Element("Carbon", "C", 6, 12.0107 * g/mole);
295   @@ -456,10 +621,13 @@
296   //Water->AddElement(elH, 2);
297   //Water->AddElement(elO, 1);
298
299 - // Liquid water at 20 degC with hydrogen bound in the molecular medium.
300 - // Using G4_WATER here would leave H under the free-gas elastic treatment.
301 + // Liquid water at 20 degC with hydrogen bound in the molecular medium
302 + // (S(alpha,beta) thermal-scattering law, see G4MPhysicsList). Using
303 + // G4_WATER or plain elH here would leave H under the free-gas elastic
304 + // treatment below 4 eV. A dedicated material is used because G4_WATER
305 + // also has the nominal Geant4 density 1.0 g/cm3 rather than 0.99820 g/cm3.
306   Water = new G4Material("Water_TS_H_of_Water",
307                           0.99820 * g/cm3,
308                           WCDMediumDensity20C(0.0),
309                           2,
310                           kStateLiquid,
311                           293.15 * kelvin,
312   @@ -468,23 +636,42 @@
313   Water->AddElement(elO, 1);
314
315
316 - // Define different PropertiesTable for different water "types"
317 - waterPT1 = new G4MaterialPropertiesTable();
318 - waterPT1->AddProperty("RINDEX", water1PhotonEnergyShort, water1RefIndex, 2);
319 - waterPT1->AddProperty("ABSELENGTH", water1PhotonEnergy, water1AbsLen, water1ArrEntries);
320 -
321 - // --- Material Properties Table from Geant4 OpNovice example.
322 - waterPT2 = new G4MaterialPropertiesTable();
323 - waterPT2->AddProperty("RINDEX", water2PhotonEnergy, water2RefIndex, water2ArrEntries);
324 - waterPT2->AddProperty("ABSELENGTH", water2PhotonEnergy, water2AbsLen, water2ArrEntries);
325 - waterPT2->AddProperty("MIEHG", water2PhotonEnergyMie, water2Mie, water2ArrEntries);
326 - waterPT2->AddConstProperty("MIEHG_FORWARD", water2MieConst[0]);
327 - waterPT2->AddConstProperty("MIEHG_BACKWARD", water2MieConst[1]);
328 - waterPT2->AddConstProperty("MIEHG_FORWARD_RATIO", water2MieConst[2]);
329 + // One complete optical table per active medium. RINDEX changes with NaCl;
330 + // ABSELENGTH and MIEHG are common baselines until concentration-dependent
331 + // measurements are available.
332 + waterPT1 = CreateWCDOpticalTable(pureWaterRefIndex);
333 + waterPT2 = waterPT1; // backward-compatible alias
334 + G4MaterialPropertiesTable* saltyWater2p5PT =
335 +   CreateWCDOpticalTable(saltyWater2p5RefIndex);
336 + G4MaterialPropertiesTable* saltyWater5PT =
337 +   CreateWCDOpticalTable(saltyWater5RefIndex);
338 + G4MaterialPropertiesTable* saltyWater10PT =
339 +   CreateWCDOpticalTable(saltyWater10RefIndex);
340
341 - // function to set waterPT from configuration file
342   Water->SetMaterialPropertiesTable(waterPT1);
343
344 + // -----
345 + // Saline water materials at 20 C
346 + // Convention: w_NaCl = m_NaCl / (m_H2O + m_NaCl).
347 + // elTSHWater (not elH) supplies hydrogen so every solution keeps the
348 + // bound-hydrogen S(alpha,beta) treatment used by pure water.
349 + // -----
350 + SaltyWater_2p5 = CreateSaltyWaterByFinalMassFraction(
351 +   SaltyWaterName(0.025), 0.025, elTSHWater, elO, elNa, elCl,
352 +   saltyWater2p5PT);
353 +
354 + SaltyWater_5 = CreateSaltyWaterByFinalMassFraction(
355 +   SaltyWaterName(0.05), 0.05, elTSHWater, elO, elNa, elCl,
356 +   saltyWater5PT);
357 +
358 + SaltyWater_10 = CreateSaltyWaterByFinalMassFraction(

```

```

359 + SaltyWaterName(0.10), 0.10, elTSHWater, elO, elNa, elCl,
360 + saltyWater10PT);
361 +
362 + // Backward-compatible default alias. Prefer explicit SaltyWater_2p5,
363 + // SaltyWater_5 or SaltyWater_10 in the detector geometry.
364 + SaltyWater = SaltyWater_10;
365 +
366
367 // -----
368 // Rocks
369 @@ -577,15 +764,14 @@
370
371
372 // -----
373 - // Borosilicate glass "Pyrex" (for SiPM & PMT active surfaces)
374 + // Effective absorbing PMT medium (keeps legacy material name "Pyrex")
375 // -----
376 /*
377 - NOTE : The "face" is actually an ellipsoidal surface
378 - The active area (photocathode) covers 90% of it.
379 - The refractive index is the same as the dome
380 - refractive index. The absorption length is set
381 - so that no photons propagate through the
382 - photocathode.
383 + This is not a resolved glass-window/vacuum/photocathode stack. The
384 + ellipsoidal solid represents an effective photocathode: Fresnel losses
385 + are obtained from its RINDEX and photons that enter are absorbed. The
386 + wavelength-dependent QE and collection efficiency are sampled exactly
387 + once by G4MPMTAction/OptDevice. Do not also add EFFICIENCY here.
388 */
389
390 // from PDG:
391 @@ -597,46 +783,89 @@
392 Pyrex->AddMaterial(B202, 0.13);
393 Pyrex->AddMaterial(Na2O, 0.07);
394
395 - // Add properties to table
396 - #warning "This properties are for PMTs! Check Pyrex optical properties for SiPMs!!"
397 + // Optical properties for the effective PMT absorber.
398 pyrexPT = new G4MaterialPropertiesTable();
399 pyrexPT->AddProperty("RINDEX", pmtPhotonEnergy, pmtRefIndex, pmtArrEntries);
400 pyrexPT->AddProperty("ABSLLENGTH", pmtPhotonEnergy, pmtAbsLength, pmtArrEntries);
401 Pyrex->SetMaterialPropertiesTable(pyrexPT);
402 -
403 +
404 // -----
405 - // HDPE for WCD liner material (Tyvek®)
406 + // Tyvek / HDPE liner material for WCD
407 // -----
408 //
409 - HDPE = new G4Material("HDPE", 0.94 * g/cm3, 2);
410 +
411 + HDPE = new G4Material("Tyvek_HDPE", 0.94 * g/cm3, 2);
412 HDPE->AddElement(elC, 2);
413 HDPE->AddElement(elH, 4);
414
415 + // Bulk optical properties of Tyvek/HDPE
416 linerPT1 = new G4MaterialPropertiesTable();
417 - linerPT1->AddProperty("ABSLLENGTH", water1PhotonEnergy, linerAbsLen, water1ArrEntries);
418 +
419 + linerPT1->AddProperty("RINDEX",
420 + water1PhotonEnergy,
421 + linerRefIndex,
422 + water1ArrEntries);
423 +
424 + linerPT1->AddProperty("ABSLLENGTH",
425 + water1PhotonEnergy,
426 + linerAbsLen,
427 + water1ArrEntries);
428 +
429 HDPE->SetMaterialPropertiesTable(linerPT1);
430
431 - // Liner optical surface properties
432 +

```

```

433 + // -----
434 + // Optical surface: water/salty water -> Tyvek
435 + // -----
436 +
437   linerOpticalPT = new G4MaterialPropertiesTable();
438   - linerOpticalPT->AddProperty("SPECULARLOBECONSTANT", specularLobePhotonEnergy, specularLobe, 3);
439   - linerOpticalPT->AddProperty("SPECULARSPIKECONSTANT", specularLobePhotonEnergy, specularSpike, 3);
440   - linerOpticalPT->AddProperty("REFLECTIVITY", water1PhotonEnergy, linerReflectivity, water1ArrEntries);
441   - linerOpticalPT->AddProperty("BACKSCATTERCONSTANT", water1PhotonEnergy, linerBackScatter, water1ArrEntries);
442   - linerOpticalPT->AddProperty("RINDEX", water1PhotonEnergy, linerRefIndex, water1ArrEntries);
443
444   - LinerOptSurf = new G4OpticalSurface("WallSurface");
445   + // Required by G4OpBoundaryProcess for a dielectric_dielectric
446   + // groundbackpainted surface. This describes the refractive index of the
447   + // optical coating at the active-medium -> Tyvek boundary.
448   + linerOpticalPT->AddProperty("RINDEX",
449   +                             water1PhotonEnergy,
450   +                             linerRefIndex,
451   +                             water1ArrEntries);
452   +
453   + linerOpticalPT->AddProperty("REFLECTIVITY",
454   +                             water1PhotonEnergy,
455   +                             linerReflectivity,
456   +                             water1ArrEntries);
457   +
458   + linerOpticalPT->AddProperty("BACKSCATTERCONSTANT",
459   +                             water1PhotonEnergy,
460   +                             linerBackScatter,
461   +                             water1ArrEntries);
462   +
463   + linerOpticalPT->AddProperty("SPECULARLOBECONSTANT",
464   +                             specularLobePhotonEnergy,
465   +                             specularLobe,
466   +                             3);
467   +
468   + linerOpticalPT->AddProperty("SPECULARSPIKECONSTANT",
469   +                             specularLobePhotonEnergy,
470   +                             specularSpike,
471   +                             3);
472   +
473   + LinerOptSurf = new G4OpticalSurface("TyvekOpticalSurface");
474   +
475   LinerOptSurf->SetModel(unified);
476   - LinerOptSurf->SetType(dielectric_metal);
477   - LinerOptSurf->SetFinish(ground);
478   + LinerOptSurf->SetType(dielectric_dielectric);
479   + LinerOptSurf->SetFinish(groundbackpainted);
480   LinerOptSurf->SetSigmaAlpha(fSigmaAlpha);
481   LinerOptSurf->SetMaterialPropertiesTable(linerOpticalPT);
482
483   - // LAGO WCD parameters
484   - LinerOptSurf2 = new G4OpticalSurface("WallSurface");
485   + // Alternate LAGO WCD liner surface (dielectric_LUT/groundtyvekair). Not
486   + // used by SaltyWCD::BuildDetector, kept initialized for parity with other
487   + // detector models that may select it directly from Materials.
488   + LinerOptSurf2 = new G4OpticalSurface("WallSurface");
489   LinerOptSurf2->SetType(dielectric_LUT);
490   LinerOptSurf2->SetFinish(groundtyvekair);
491   LinerOptSurf2->SetModel(LUT);
492
493   +
494   +
495   +
496   // -----
497   // Concrete
498   // -----

```

A.5. G4Models/SaltyWCD.cc

```

1 --- MEIGA-anterior/src/G4Models/SaltyWCD.cc
2 +++ MEIGA-integrado/src/G4Models/SaltyWCD.cc
3 @@ -6,253 +6,461 @@

```

```

4  #include "G4MDetectorAction.h"
5  #include "G4MPMTAction.h"
6
7  -
8  #include "G4VisAttributes.hh"
9  #include "G4NistManager.hh"
10 #include "G4Colour.hh"
11 +#include "G4Tubs.hh"
12 +#include "G4Ellipsoid.hh"
13 +#include "G4PVPlacement.hh"
14 +#include "G4LogicalVolume.hh"
15 +#include "G4LogicalBorderSurface.hh"
16 +#include "G4SDManager.hh"
17 +#include "G4SystemOfUnits.hh"
18 #include "G4Exception.hh"
19 +#include "G4MaterialPropertiesTable.hh"
20 +#include "G4OpticalSurface.hh"
21
22 using namespace std;
23
24 namespace
25 {
26 -G4double NormalizeNaClMassFraction(G4double value)
27 -{
28 - if (value > 1.0) value /= 100.0;
29 - return value;
30 + // This helper is only for selecting one of the materials already built in
31 + // Materials.cc. It does NOT create a new chemical material here.
32 + G4double NormalizeDetectorNaClInput(G4double x)
33 + {
34 + // Accept 0.025 and 2.5 as the same input convention.
35 + if (x > 1.0) x /= 100.0;
36 +
37 + return x;
38 + }
39 +
40 + G4Material* SelectWCDMedium(Materials& mat, G4double userNaClValue)
41 + {
42 + // Final mass fraction:  $w_{NaCl} = m_{NaCl} / (m_{H2O} + m_{NaCl})$ .
43 + const G4double w = NormalizeDetectorNaClInput(userNaClValue);
44 + const G4double tol = 1.0e-6;
45 +
46 + if (std::abs(w) < tol) {
47 + return mat.Water;
48 + }
49 +
50 + if (std::abs(w - 0.025) < tol) {
51 + return mat.SaltyWater_2p5;
52 + }
53 + if (std::abs(w - 0.050) < tol) {
54 + return mat.SaltyWater_5;
55 + }
56 + if (std::abs(w - 0.100) < tol) {
57 + return mat.SaltyWater_10;
58 + }
59 +
60 + G4ExceptionDescription description;
61 + description << "Unsupported WCD NaCl final mass fraction: " << w
62 + << ". Use 0, 0.025, 0.050 or 0.100 "
63 + << "(equivalently 0, 2.5, 5 or 10 percent).";
64 + G4Exception("SelectWCDMedium", "WCD-MAT-002",
65 + FatalException, description);
66 + return nullptr;
67 + }
68 }
69
70 -G4double SalineDensity20C(G4double wNaCl)
71 +void
72 +SaltyWCD::BuildDetector(G4LogicalVolume* logMother,
73 + Detector& detector,
74 + Event& theEvent,
75 + G4bool fCheckOverlaps)
76 + {
77 - const G4double tolerance = 1.0e-9;

```

```

78 - if (std::abs(wNaCl - 0.000) < tolerance) return 0.99820*g/cm3;
79 - if (std::abs(wNaCl - 0.025) < tolerance) return 1.01604*g/cm3;
80 - if (std::abs(wNaCl - 0.050) < tolerance) return 1.03401*g/cm3;
81 - if (std::abs(wNaCl - 0.100) < tolerance) return 1.07051*g/cm3;
82 -
83 - G4ExceptionDescription description;
84 - description << "Unsupported final NaCl mass fraction " << wNaCl
85 -           << ". Use 0, 0.025, 0.050 or 0.100.";
86 - G4Exception("SalineDensity20C", "MEIGA-TSL-002",
87 -           FatalException, description);
88 - return 0.0;
89 -}
90 -}
91 + // -----
92 + // Materials
93 + // -----
94 + Materials mat;
95 +
96 + G4NistManager* nist = G4NistManager::Instance();
97 + G4Material* StainlessSteel =
98 +     nist->FindOrBuildMaterial("G4_STAINLESS-STEEL");
99 +
100 + const G4double fNaClFracMass = detector.GetImpuritiesFraction();
101 + G4Material* DetectorMedium = SelectWCDMedium(mat, fNaClFracMass);
102 +
103 + // -----
104 + // WCD dimensions
105 + // -----
106 + // In this geometry detector.GetTankRadius() and detector.GetTankHeight()
107 + // are interpreted as the active water/saline-water dimensions.
108 + //
109 + // Active volume:
110 + //   diameter = 96 cm -> radius = 48 cm
111 + //   height   = 133 cm
112 + //
113 + // Layer sequence:
114 + //   active water/saline water -> Tyvek liner -> stainless-steel casing
115 + // -----
116 + const G4double fWaterRadius      = detector.GetTankRadius();
117 + const G4double fWaterHeight      = detector.GetTankHeight();
118 + const G4double fWaterHalfHeight = 0.5 * fWaterHeight;
119 +
120 + const G4double fSteelThickness  = detector.GetTankThickness();
121 + const G4double fTyvekThickness  = 0.12 * mm;
122 +
123 + const G4double fTyvekOuterRadius = fWaterRadius + fTyvekThickness;
124 + const G4double fSteelOuterRadius = fTyvekOuterRadius + fSteelThickness;
125 +
126 + const G4double fTyvekSideHalfHeight = fWaterHalfHeight;
127 + const G4double fSteelSideHalfHeight = fWaterHalfHeight + fTyvekThickness;
128 +
129 + // -----
130 + // PMT properties
131 + // -----
132 + OptDevice pmt = detector.GetOptDevice(OptDevice::ePMT);
133 +
134 + const G4double fPMTSemiX = pmt.GetSemiAxisX() * CLHEP::cm;
135 + const G4double fPMTSemiY = pmt.GetSemiAxisY() * CLHEP::cm;
136 + const G4double fPMTSemiZ = pmt.GetSemiAxisZ() * CLHEP::cm;
137 +
138 + // -----
139 + // Detector position
140 + // -----
141 + G4ThreeVector detectorPos = Geometry::ToG4Vector(detector.GetDetectorPosition(), 1.);
142 +
143 + const G4double fTankPosX = detectorPos.getX();
144 + const G4double fTankPosY = detectorPos.getY();
145 + const G4double fTankPosZ = detectorPos.getZ();
146 +
147 + // Bottom reference is fTankPosZ. The active water volume starts after
148 + // the bottom steel plate and the bottom Tyvek sheet.
149 + const G4double zSteelBot  = fTankPosZ + 0.5*fSteelThickness;
150 + const G4double zTyvekBot  = fTankPosZ + fSteelThickness + 0.5*fTyvekThickness;
151 + const G4double zWater     = fTankPosZ + fSteelThickness + fTyvekThickness

```

```

152 +         + fWaterHalfHeight;
153 + const G4double zTyvekTop = fTankPosZ + fSteelThickness + fTyvekThickness
154 +         + fWaterHeight + 0.5*fTyvekThickness;
155 + const G4double zSteelTop = fTankPosZ + fSteelThickness + fTyvekThickness
156 +         + fWaterHeight + fTyvekThickness
157 +         + 0.5*fSteelThickness;
158 +
159 + const G4ThreeVector waterCenter(fTankPosX, fTankPosY, zWater);
160 +
161 + const int detectorId = detector.GetId();
162 + const int pmtId = 0;
163 +
164 + ostringstream namedetector;
165 + namedetector << "/SaltyWCD" << detectorId;
166 +
167 + G4cout << "[INFO] G4Models::SaltyWCD: Building detector "
168 +         << namedetector.str()
169 +         << " (ID = " << detectorId << ") with "
170 +         << pmt.GetName() << "." << G4endl;
171 +
172 + G4cout << "[INFO] G4Models::SaltyWCD: Active medium = "
173 +         << DetectorMedium->GetName() << G4endl;
174 + G4cout << "[INFO] G4Models::SaltyWCD: NaCl final mass fraction = "
175 +         << NormalizedDetectorNaClInput(fNaClFracMass) << G4endl;
176 + G4cout << "[INFO] G4Models::SaltyWCD: Active-medium density = "
177 +         << DetectorMedium->GetDensity() / (g/cm3) << " g/cm3" << G4endl;
178 + G4MaterialPropertiesTable* opticalTable =
179 +     DetectorMedium->GetMaterialPropertiesTable();
180 +
181 + if (!opticalTable ||
182 +     !opticalTable->GetProperty("RINDEX") ||
183 +     !opticalTable->GetProperty("ABSLENGTH")) {
184 +     G4Exception(
185 +         "SaltyWCD::BuildDetector",
186 +         "WCD-OPT-001",
187 +         FatalException,
188 +         "The selected WCD medium requires RINDEX and ABSLENGTH.");
189 + }
190 +
191 + G4MaterialPropertiesTable* tyvekSurfaceTable =
192 +     mat.LinerOptSurf
193 +     ? mat.LinerOptSurf->GetMaterialPropertiesTable()
194 +     : nullptr;
195 +
196 + if (!tyvekSurfaceTable ||
197 +     !tyvekSurfaceTable->GetProperty("RINDEX") ||
198 +     !tyvekSurfaceTable->GetProperty("REFLECTIVITY")) {
199 +     G4Exception(
200 +         "SaltyWCD::BuildDetector",
201 +         "WCD-OPT-002",
202 +         FatalException,
203 +         "The Tyvek groundbackpainted surface requires RINDEX and REFLECTIVITY.");
204 + }
205 +
206 + G4MaterialPropertiesTable* pmtTable =
207 +     mat.Pyrex ? mat.Pyrex->GetMaterialPropertiesTable() : nullptr;
208 + if (!pmtTable ||
209 +     !pmtTable->GetProperty("RINDEX") ||
210 +     !pmtTable->GetProperty("ABSLENGTH")) {
211 +     G4Exception(
212 +         "SaltyWCD::BuildDetector",
213 +         "WCD-OPT-003",
214 +         FatalException,
215 +         "The effective PMT medium requires RINDEX and ABSLENGTH.");
216 + }
217 +
218 + G4cout << "[INFO] G4Models::SaltyWCD: Active-medium optical properties "
219 +         << "RINDEX/ABSLENGTH = assigned" << G4endl;
220 + G4cout << "[INFO] G4Models::SaltyWCD: Tyvek surface properties "
221 +         << "RINDEX/REFLECTIVITY = assigned" << G4endl;
222 + G4cout << "[INFO] G4Models::SaltyWCD: Effective PMT response = "
223 +         << "Fresnel RINDEX + terminal absorption + OptDevice QE" << G4endl;
224 + G4cout << "[INFO] G4Models::SaltyWCD: Water radius = "
225 +         << fWaterRadius / CLHEP::cm << " cm" << G4endl;

```

```

226 + G4cout << "[INFO] G4Models::SaltyWCD: Water height = "
227 + << fWaterHeight / CLHEP::cm << " cm" << G4endl;
228 + G4cout << "[INFO] G4Models::SaltyWCD: Tyvek thickness = "
229 + << fTyvekThickness / CLHEP::mm << " mm" << G4endl;
230 + G4cout << "[INFO] G4Models::SaltyWCD: Steel thickness = "
231 + << fSteelThickness / CLHEP::mm << " mm" << G4endl;
232 +
233 + /*****
234 + Geometry model
235 +
236 + The WCD is built as explicit, non-overlapping layers:
237 +
238 + 1. Active volume:
239 + pure water or predefined saline water material:
240 + Water, SaltyWater_2p5, SaltyWater_5, SaltyWater_10.
241 +
242 + 2. Tyvek liner:
243 + material Materials::HDPE, named Tyvek_HDPE in Materials.cc.
244 + Its optical behavior is controlled by Materials::LinerOptSurf.
245 +
246 + 3. Stainless-steel external casing:
247 + G4_STAINLESS-STEEL from NIST.
248 +
249 + 4. PMT:
250 + ellipsoidal Pyrex window placed inside the active volume and
251 + registered as a sensitive detector.
252 + *****/
253 +
254 + // -----
255 + // Solids
256 + // -----
257 + G4Tubs* solidWater = new G4Tubs("WaterVolume",
258 + 0.0,
259 + fWaterRadius,
260 + fWaterHalfHeight,
261 + 0.0,
262 + 360.0*deg);
263 +
264 + G4Tubs* solidTyvekBot = new G4Tubs("TyvekBottom",
265 + 0.0,
266 + fTyvekOuterRadius,
267 + 0.5*fTyvekThickness,
268 + 0.0,
269 + 360.0*deg);
270 +
271 + G4Tubs* solidTyvekTop = new G4Tubs("TyvekTop",
272 + 0.0,
273 + fTyvekOuterRadius,
274 + 0.5*fTyvekThickness,
275 + 0.0,
276 + 360.0*deg);
277 +
278 + G4Tubs* solidTyvekSide = new G4Tubs("TyvekSide",
279 + fWaterRadius,
280 + fTyvekOuterRadius,
281 + fTyvekSideHalfHeight,
282 + 0.0,
283 + 360.0*deg);
284 +
285 + G4Tubs* solidSteelBot = new G4Tubs("SteelBottom",
286 + 0.0,
287 + fSteelOuterRadius,
288 + 0.5*fSteelThickness,
289 + 0.0,
290 + 360.0*deg);
291 +
292 + G4Tubs* solidSteelTop = new G4Tubs("SteelTop",
293 + 0.0,
294 + fSteelOuterRadius,
295 + 0.5*fSteelThickness,
296 + 0.0,
297 + 360.0*deg);
298 +
299 + G4Tubs* solidSteelSide = new G4Tubs("SteelSide",

```

```

300 +         fTyvekOuterRadius,
301 +         fSteelOuterRadius,
302 +         fSteelSideHalfHeight,
303 +         0.0,
304 +         360.0*deg);
305 +
306 + G4Ellipsoid* solidPMT = new G4Ellipsoid("PMT",
307 +         fPMTSemiX,
308 +         fPMTSemiY,
309 +         fPMTSemiZ,
310 +         -fPMTSemiZ,
311 +         0.0);
312 +
313 + // -----
314 + // Logical volumes
315 + // -----
316 + G4LogicalVolume* logTank =
317 +     new G4LogicalVolume(solidWater, DetectorMedium, "logTank", 0, 0, 0);
318 +
319 + G4LogicalVolume* logTyvekBot =
320 +     new G4LogicalVolume(solidTyvekBot, mat.HDPE, "logTyvekBot", 0, 0, 0);
321 +
322 + G4LogicalVolume* logTyvekTop =
323 +     new G4LogicalVolume(solidTyvekTop, mat.HDPE, "logTyvekTop", 0, 0, 0);
324 +
325 + G4LogicalVolume* logTyvekSide =
326 +     new G4LogicalVolume(solidTyvekSide, mat.HDPE, "logTyvekSide", 0, 0, 0);
327 +
328 + G4LogicalVolume* logCasingBot =
329 +     new G4LogicalVolume(solidSteelBot, StainlessSteel, "logCasingBot", 0, 0, 0);
330 +
331 + G4LogicalVolume* logCasingTop =
332 +     new G4LogicalVolume(solidSteelTop, StainlessSteel, "logCasingTop", 0, 0, 0);
333 +
334 + G4LogicalVolume* logCasingSide =
335 +     new G4LogicalVolume(solidSteelSide, StainlessSteel, "logCasingSide", 0, 0, 0);
336 +
337 + // -----
338 + // Physical placements
339 + // -----
340 + G4PVPlacement* physTank =
341 +     new G4PVPlacement(nullptr,
342 +         waterCenter,
343 +         logTank,
344 +         "physTank",
345 +         logMother,
346 +         false,
347 +         0,
348 +         fCheckOverlaps);
349 +
350 + G4PVPlacement* physTyvekBot =
351 +     new G4PVPlacement(nullptr,
352 +         G4ThreeVector(fTankPosX, fTankPosY, zTyvekBot),
353 +         logTyvekBot,
354 +         "physTyvekBot",
355 +         logMother,
356 +         false,
357 +         0,
358 +         fCheckOverlaps);
359 +
360 + G4PVPlacement* physTyvekTop =
361 +     new G4PVPlacement(nullptr,
362 +         G4ThreeVector(fTankPosX, fTankPosY, zTyvekTop),
363 +         logTyvekTop,
364 +         "physTyvekTop",
365 +         logMother,
366 +         false,
367 +         0,
368 +         fCheckOverlaps);
369 +
370 + G4PVPlacement* physTyvekSide =
371 +     new G4PVPlacement(nullptr,
372 +         waterCenter,
373 +         logTyvekSide,

```

```

374 +         "physTyvekSide",
375 +         logMother,
376 +         false,
377 +         0,
378 +         fCheckOverlaps);
379 +
380 +     new G4PVPlacement(nullptr,
381 +         G4ThreeVector(fTankPosX, fTankPosY, zSteelBot),
382 +         logCasingBot,
383 +         "physCasingBot",
384 +         logMother,
385 +         false,
386 +         0,
387 +         fCheckOverlaps);
388 +
389 +     new G4PVPlacement(nullptr,
390 +         G4ThreeVector(fTankPosX, fTankPosY, zSteelTop),
391 +         logCasingTop,
392 +         "physCasingTop",
393 +         logMother,
394 +         false,
395 +         0,
396 +         fCheckOverlaps);
397 +
398 +     new G4PVPlacement(nullptr,
399 +         waterCenter,
400 +         logCasingSide,
401 +         "physCasingSide",
402 +         logMother,
403 +         false,
404 +         0,
405 +         fCheckOverlaps);
406 +
407 +     // -----
408 +     // Optical border surfaces: active medium -> Tyvek
409 +     // -----
410 +     new G4LogicalBorderSurface("WaterTyvekTopSurface",
411 +         physTank,
412 +         physTyvekTop,
413 +         mat.LinerOptSurf);
414 +
415 +     new G4LogicalBorderSurface("WaterTyvekBottomSurface",
416 +         physTank,
417 +         physTyvekBot,
418 +         mat.LinerOptSurf);
419 +
420 +     new G4LogicalBorderSurface("WaterTyvekSideSurface",
421 +         physTank,
422 +         physTyvekSide,
423 +         mat.LinerOptSurf);
424 +
425 +     // -----
426 +     // Effective PMT photocathode. The solid is intentionally absorbing;
427 +     // G4MPMTAction samples OptDevice's wavelength-dependent response once at
428 +     // the water/salty-water -> PMT entry boundary and terminates the photon.
429 +     // -----
430 +     string logName = "logPMT_" + to_string(pmtId);
431 +
432 +     G4LogicalVolume* logPMT =
433 +         new G4LogicalVolume(solidPMT, mat.Pyrex, logName, 0, 0, 0);
434 +
435 +     // PMT centered in x,y and located at the upper part of the active volume.
436 +     // The ellipsoid is truncated between -fPMTSemiZ and 0.
437 +     // Therefore, its flat face is located at z = zPMT.
438 +     // A small offset avoids numerical problems due to exact boundary contact.
439 +     const G4double zPMT = fWaterHalfHeight - 0.1*mm;
440 +
441 +     new G4PVPlacement(nullptr,
442 +         G4ThreeVector(0.0, 0.0, zPMT),
443 +         logPMT,
444 +         "physPMT",
445 +         logTank,
446 +         false,
447 +         pmtId,

```

```

448 +         fCheckOverlaps);
449 +
450 +         // -----
451 +         // Register volumes and sensitive detectors
452 +         // -----
453 +         if (!detector.HasLogicalVolume("logTank")) {
454 +             detector.SetLogicalVolume("logTank", logTank);
455 +         }
456 +
457 +         G4SDManager* const sdMan = G4SDManager::GetSDMpointer();
458 +
459 +         if (!detector.HasOptDevice(pmtId)) {
460 +             detector.MakeOptDevice(pmtId, OptDevice::ePMT);
461 +             G4cout << "[DEBUG] Adding PMT id = " << pmtId << G4endl;
462 +         }
463 +
464 +         OptDevice optDevice = detector.GetOptDevice(pmtId);
465 +         G4cout << "[DEBUG] Getting " << optDevice.GetName()
466 +             << " with id " << optDevice.GetId() << G4endl;
467 +
468 +         string optName = pmt.GetName() + "_" + to_string(pmtId);
469 +
470 +         ostringstream fullName;
471 +         fullName << "/SaltyWCD_" << detectorId << "/" << optName;
472 +
473 +         G4MPMTAction* const pmtSD =
474 +             new G4MPMTAction(fullName.str().c_str(), detectorId, pmtId, theEvent);
475 +
476 + -void
477 + -SaltyWCD::BuildDetector(G4LogicalVolume* logMother, Detector& detector, Event& theEvent, G4bool
         fCheckOverlaps)
478 + -{
479 +     sdMan->AddNewDetector(pmtSD);
480 +     logPMT->SetSensitiveDetector(pmtSD);
481 +
482 +     G4MDetectorAction* const waterSD =
483 +         new G4MDetectorAction(namedetector.str().c_str(), detectorId, theEvent);
484 +
485 + -
486 + - // solids
487 + - G4Tubs* solidCasingTop = nullptr;
488 + - G4Tubs* solidCasingSide = nullptr;
489 + -
490 + - G4Tubs* solidTank = nullptr;
491 + - G4Tubs* solidTop = nullptr;
492 + - G4Tubs* solidBot = nullptr;
493 + - G4Tubs* solidSide = nullptr;
494 + -
495 + - // pmt solids
496 + - G4Ellipsoid* solidPMT = nullptr;
497 + - //G4Sphere* solidPMT = nullptr;
498 + - //G4Sphere* solidSphere = nullptr;
499 + -
500 + - // logical volumes
501 + - G4LogicalVolume* logCasingTop = nullptr;
502 + - G4LogicalVolume* logCasingBot = nullptr;
503 + - G4LogicalVolume* logCasingSide = nullptr;
504 + -
505 + - G4LogicalVolume* logTank = nullptr;
506 + - G4LogicalVolume* logTop = nullptr;
507 + - G4LogicalVolume* logBot = nullptr;
508 + - G4LogicalVolume* logSide = nullptr;
509 + -
510 + - G4LogicalVolume* logPMT = nullptr;
511 + -
512 + -
513 + - // physical volumes
514 + - G4PVPlacement* physCasingBot = nullptr;
515 + - G4PVPlacement* physCasingTop = nullptr;
516 + - G4PVPlacement* physCasingSide = nullptr;
517 + -
518 + - G4PVPlacement* physTank = nullptr;
519 + - G4PVPlacement* physBot = nullptr;
520 + - G4PVPlacement* physTop = nullptr;

```

```

521 - G4PVPlacement* physSide = nullptr;
522 -
523 - // SaltyWCD dimensions
524 - G4double fTankRadius = detector.GetTankRadius();
525 - G4double fTankHeight = detector.GetTankHeight();
526 - G4double fTankHalfHeight = 0.5 * fTankHeight;
527 - G4double fTankThickness = detector.GetTankThickness();
528 - G4double fNaClFracMass =
529 -     NormalizeNaClMassFraction(detector.GetImpuritiesFraction());
530 - // -----
531 - // SaltyWater defined here for this particular detector
532 - // -----
533 - // Materials().Water contains TS_H_of_Water, so H in every NaCl mixture
534 - // uses the h_water thermal-scattering law. Na, Cl and O retain their
535 - // standard G4NDL treatment; G4NDL has no concentration-specific saline TSL.
536 - G4Material* SaltyWater = new G4Material("SaltyWater_TS_H_of_Water",
537 -     SalineDensity20C(fNaClFracMass),
538 -     2,
539 -     kStateLiquid,
540 -     293.15 * kelvin,
541 -     1.0 * bar);
542 - SaltyWater->AddMaterial(Materials().Water, 1-fNaClFracMass);
543 - SaltyWater->AddMaterial(Materials().Salt, fNaClFracMass);
544 - SaltyWater->SetMaterialPropertiesTable(Materials().waterPT1);
545 -
546 - G4NistManager* nist = G4NistManager::Instance();
547 - G4Material* StainlessSteel = nist->FindOrBuildMaterial("G4_STAINLESS-STEEL");
548 -
549 - // PMT properties photonis-XP1805
550 - OptDevice pmt = detector.GetOptDevice(OptDevice::ePMT);
551 - //G4double fPMTinnerRadius = 0.0 * cm; // Radio interno (0 para esfera sólida)
552 - //G4double fPMTouterRadius = 10.0 * cm; // Radio externo (10 cm)
553 - //G4double fPMTstartPhi = 0.0 * deg; // Ángulo Phi inicial
554 - //G4double fPMTdeltaPhi = 360.0 * deg; // Ángulo Phi final (completa)
555 - //G4double fPMTstartTheta = 0.0 * deg; // Ángulo Theta inicial
556 - //G4double fPMTdeltaTheta = 180.0 * deg; // Ángulo Theta final (completa)
557 -
558 - //G4double fPMTinnerRad = pmt.GetinnerRadius() * CLHEP::cm;
559 - //G4double fPMTouterRad = pmt.GetouterRadius() * CLHEP::cm;
560 - //G4double fPMTstartP = pmt.GetstartPhi() * CLHEP::deg;
561 - //G4double fPMTdeltaP = pmt.GetdeltaPhi() * CLHEP::deg;
562 - //G4double fPMTstartTh = pmt.GetstartTheta() * CLHEP::deg;
563 - //G4double fPMTdeltaTh = pmt.GetdeltaTheta() * CLHEP::deg;
564 - G4double fPMTSemiX = pmt.GetSemiAxisX() * CLHEP::cm;
565 - G4double fPMTSemiY = pmt.GetSemiAxisY() * CLHEP::cm;
566 - G4double fPMTSemiZ = pmt.GetSemiAxisZ() * CLHEP::cm;
567 -
568 - G4ThreeVector detectorPos = Geometry::ToG4Vector(detector.GetDetectorPosition(), 1.);
569 - G4double fTankPosX = detectorPos.getX();
570 - G4double fTankPosY = detectorPos.getY();
571 - G4double fTankPosZ = detectorPos.getZ();
572 -
573 - // define PMT position as the center of the tank
574 - //G4ThreeVector fTankCenter = detectorPos + G4ThreeVector(0, 50*cm, fTankHalfHeight + fTankThickness);
575 - G4ThreeVector fTankCenter = detectorPos + G4ThreeVector(0,0, fTankHalfHeight + fTankThickness);
576 - int detectorId = detector.GetId();
577 - int pmtId = 0;
578 - ostream namedetector;
579 - namedetector.str("");
580 - namedetector << "/SaltyWCD"+to_string(detectorId);
581 - cout << "[INFO] G4Models::SaltyWCD: Building detector " << namedetector.str();
582 - cout << " (ID = " << detectorId << ")";
583 - cout << " with " << pmt.GetName() << ". " << endl;
584 - cout << "[INFO] G4Models::SaltyWCD: Detector Dimensions:" << endl;
585 - cout << "Tank Radius = " << fTankRadius / CLHEP::cm << " cm " << endl;
586 - cout << "Tank Height = " << fTankHeight / CLHEP::cm << " cm " << endl;
587 - cout << "Fraccion de impurezas en el agua = " << fNaClFracMass << endl;
588 - /*****
589 -
590 - Geant4 Volume construction
591 -
592 - SaltyWCD is a WCD filled with salty water. The fraction of
593 - NaCl w.r.t water is determined by the parameter fNaClFracMass
594 - which value is between 0 and 1.

```

```

595 -
596 -   The water is contained in a cylinder of stainless steel.
597 -
598 -   The PMT is simulated as a semi-sphere made of Pyrex
599 -   to account for reflectivity of photons at the PMT
600 -   window. The PMT volume is registered as a SensitiveDetector
601 -   and its response is computed by the G4MPMTAction class.
602 -
603 -   *****/
604 - solidCasingTop = new G4Tubs("CasingTop", 0, fTankRadius + fTankThickness, fTankThickness/2, 0, 360*deg);
605 - solidCasingSide = new G4Tubs("CasingSide", fTankRadius, fTankRadius + fTankThickness, fTankHalfHeight, 0,
606 -   360*deg);
607 -
608 - solidTank = new G4Tubs("Tank", 0, fTankRadius, fTankHalfHeight, 0, 360*deg);
609 - solidTop = new G4Tubs("Top", 0, fTankRadius, fTankThickness/2, 0, 360*deg);
610 - solidSide = new G4Tubs("Side", fTankRadius, fTankRadius + fTankThickness, fTankHalfHeight, 0, 360*deg);
611 -
612 - // pmt solids
613 - solidPMT = new G4Ellipsoid("PMT", fPMTSemiX, fPMTSemiY, fPMTSemiZ, -fPMTSemiZ, 0);
614 - //solidPMT = new G4Sphere("PMT", fPMTInnerRad, fPMTOuterRad, fPMTstartP, fPMTdeltaP, fPMTstartTh,
615 -   fPMTdeltaTh);
616 -
617 - // assemble SaltyWCD
618 - G4SDManager* const sdMan = G4SDManager::GetSDMpointer();
619 -
620 - // tank casing are made of Stainless-steel
621 - logCasingTop = new G4LogicalVolume(solidCasingTop, StainlessSteel, "logCasingTop", 0, 0, 0);
622 - physCasingTop = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, fTankPosY, fTankPosZ + 2*
623 -   fTankHalfHeight + 1.5*fTankThickness), logCasingTop, "physCasingTop", logMother, false, 0,
624 -   fCheckOverlaps);
625 - //physCasingTop = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, -100*cm, fTankPosZ + 2*
626 -   fTankHalfHeight + 1.5*fTankThickness), logCasingTop, "physCasingTop", logMother, false, 0,
627 -   fCheckOverlaps);
628 -
629 - logCasingBot = new G4LogicalVolume(solidCasingTop, StainlessSteel, "logCasingBot", 0, 0, 0);
630 - physCasingBot = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, fTankPosY, fTankPosZ + 0.5*
631 -   fTankThickness), logCasingBot, "physCasingBot", logMother, false, 0);
632 - //physCasingBot = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, -100*cm, fTankPosZ + 0.5*
633 -   fTankThickness), logCasingBot, "physCasingBot", logMother, false, 0);
634 -
635 - logCasingSide = new G4LogicalVolume(solidCasingSide, StainlessSteel, "logCasingSide", 0, 0, 0);
636 - physCasingSide = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, fTankPosY, fTankPosZ +
637 -   fTankHalfHeight + fTankThickness), logCasingSide, "physCasingSide", logMother, false, 0,
638 -   fCheckOverlaps);
639 - //physCasingSide = new G4PVPlacement(nullptr, G4ThreeVector(fTankPosX, -100*cm, fTankPosZ +
640 -   fTankHalfHeight + fTankThickness), logCasingSide, "physCasingSide", logMother, false, 0, fCheckOverlaps
641 -   );
642 -
643 - // water part
644 - logTank = new G4LogicalVolume(solidTank, SaltyWater, "logTank", 0, 0, 0);
645 - physTank = new G4PVPlacement(nullptr, fTankCenter, logTank, "physTank", logMother, false, 0,
646 -   fCheckOverlaps);
647 - // register water logical volume in the Detector
648 - if (!detector.HasLogicalVolume("logTank"))
649 -   detector.SetLogicalVolume("logTank", logTank);
650 -
651 - // top, bottom and side walls of the tank
652 - logTop = new G4LogicalVolume(solidTop, Materials().HDPE, "logTop", 0, 0, 0);
653 - physTop = new G4PVPlacement(nullptr, G4ThreeVector(), logTop, "physTop", logCasingTop, false, 0,
654 -   fCheckOverlaps);
655 - //physTop = new G4PVPlacement(nullptr, G4ThreeVector(0,0,50*cm), logTop, "physTop", logCasingTop, false,
656 -   0, fCheckOverlaps);
657 - //physTop = new G4PVPlacement(nullptr, fTankCenter, logTop, "physTop", logCasingTop, false, 0,
658 -   fCheckOverlaps);
659 -
660 - logBot = new G4LogicalVolume(solidTop, Materials().HDPE, "logBot", 0, 0, 0);
661 - physBot = new G4PVPlacement(nullptr, G4ThreeVector(), logBot, "physBot", logCasingBot, false, 0,
662 -   fCheckOverlaps);
663 - //physBot = new G4PVPlacement(nullptr, G4ThreeVector(0,0,-50*cm), logBot, "physBot", logCasingBot, false,
664 -   0, fCheckOverlaps);
665 - //physBot = new G4PVPlacement(nullptr, fTankCenter, logBot, "physBot", logCasingBot, false, 0,
666 -   fCheckOverlaps);
667 -

```

```

650 - logSide = new G4LogicalVolume(solidSide, Materials().HDPE, "logSide", 0, 0, 0);
651 - physSide = new G4PVPlacement(nullptr, G4ThreeVector(), logSide, "physSide", logCasingSide, false, 0,
    fCheckOverlaps);
652 - //physSide = new G4PVPlacement(nullptr, G4ThreeVector(0,50*cm,0), logSide, "physSide", logCasingSide,
    false, 0, fCheckOverlaps);
653 -
654 - // tank liner surface
655 - new G4LogicalBorderSurface("TopSurface", physTank, physTop, Materials().LinerOptSurf);
656 - new G4LogicalBorderSurface("BotSurface", physTank, physBot, Materials().LinerOptSurf);
657 - new G4LogicalBorderSurface("SideSurface", physTank, physSide, Materials().LinerOptSurf);
658 -
659 - // PMT inicial sin cambios
660 - //string logName = "logPMT_"+to_string(pmtId);
661 - //logPMT = new G4LogicalVolume(solidPMT, Materials().Pyrex, logName, 0, 0, 0);
662 - //new G4PVPlacement(nullptr, G4ThreeVector(0, 0, fTankHalfHeight), logPMT, "physPMT", logTank, false,
    pmtId, fCheckOverlaps);
663 -
664 - // PMT
665 - string logName = "logPMT_"+to_string(pmtId);
666 - logPMT = new G4LogicalVolume(solidPMT, Materials().Pyrex, logName, 0, 0, 0);
667 - //new G4PVPlacement(nullptr, G4ThreeVector(0, 0, 0.8*fTankHalfHeight), logPMT, "physPMT", logTank, false,
    pmtId, fCheckOverlaps);
668 - //new G4PVPlacement(nullptr, G4ThreeVector(0, 0, 0.4*fTankHalfHeight), logPMT, "physPMT", logTank, false,
    pmtId, fCheckOverlaps);
669 - new G4PVPlacement(nullptr, G4ThreeVector(0, 0, 0.8*fTankHalfHeight), logPMT, "physPMT", logTank, false,
    pmtId, fCheckOverlaps);
670 -
671 -
672 - // register PMT in the Detector
673 - if (!detector.HasOptDevice(pmtId)) {
674 -     detector.MakeOptDevice(pmtId, OptDevice::ePMT);
675 -     cout << "[DEBUG] Adding PMT id = " << pmtId << endl;
676 - }
677 - OptDevice optDevice = detector.GetOptDevice(pmtId);
678 - cout << "[DEBUG] Getting " << optDevice.GetName() << " with id " << optDevice.GetId() << endl;
679 - // register PMT logical volume
680 - //if (!optDevice.HasLogicalVolume(logName))
681 - //optDevice.SetLogicalVolume(logName, logPMT);
682 -
683 - string optName = pmt.GetName() + "_" + to_string(pmtId);
684 - ostringstream fullName;
685 - fullName.str("");
686 - fullName << "/SaltyWCD/" + to_string(detectorId) << "/" << optName;
687 - G4MPMTAction* const pmtSD = new G4MPMTAction(fullName.str().c_str(), detectorId, pmtId, theEvent);
688 - sdMan->AddNewDetector(pmtSD);
689 - logPMT->SetSensitiveDetector(pmtSD);
690 -
691 -
692 - // register water volume as sensitive detector
693 - G4MDetectorAction* const waterSD = new G4MDetectorAction(namedetector.str().c_str(), detectorId, theEvent
    );
694 - sdMan->AddNewDetector(waterSD);
695 - logTank->SetSensitiveDetector(waterSD);
696 -
697 +     sdMan->AddNewDetector(waterSD);
698 +     logTank->SetSensitiveDetector(waterSD);
699 }

```

A.6. G4Models/G4MPMTAction.h

```

1 --- MEIGA-anterior/src/G4Models/G4MPMTAction.h
2 +++ MEIGA-integrado/src/G4Models/G4MPMTAction.h
3 @@ -24,9 +24,9 @@
4
5     int fDetectorId = 0;
6     int fOptDeviceId = 0;
7     int NumCerenkovPhotons;
8     int NumPE;
9     int NumPEMuDecay;
10 +    int NumCerenkovPhotons = 0;
11 +    int NumPE = 0;
12 +    int NumPEMuDecay = 0;
13

```

```

14     std::vector<double> fPETime;
15     std::vector<double> fPETimeComp;
16 @@ -35,4 +35,4 @@
17 };
18
19
20 -#endif
21 \ No hay ningún carácter de nueva línea al final del archivo
22 +#endif

```

A.7. G4Models/G4MPMTAction.cc

```

1  --- MEIGA-anterior/src/G4Models/G4MPMTAction.cc
2  +++ MEIGA-integrado/src/G4Models/G4MPMTAction.cc
3  @@ -31,8 +31,12 @@
4  void
5  G4MPMTAction::Initialize(G4HCofThisEvent* const /*hce*/)
6  {
7  -
8  -
9  + NumCerenkovPhotons = 0;
10 + NumPE = 0;
11 + NumPEMuDecay = 0;
12 + fPETime.clear();
13 + fPETimeComp.clear();
14 + fPETimeMuDecay.clear();
15 }
16
17 void
18 @@ -89,53 +93,52 @@
19 G4bool
20 G4MPMTAction::ProcessHits(G4Step* const step, G4TouchableHistory* const /*rOHist*/)
21 {
22 -
23 - // reject particle in case it is not a photon
24 - if (step->GetTrack()->GetDefinition() != G4OpticalPhoton::OpticalPhotonDefinition())
25 -     return true;
26 + G4Track* const track = step->GetTrack();
27
28 - // for RICH paper: get ParentID of photons
29 - int parentId = step->GetTrack()->GetParentID();
30 - // The PMT solid is an effective absorbing photocathode. Its response must
31 - // be sampled exactly once, when an optical photon enters the volume.
32 + if (track->GetDefinition() != G4OpticalPhoton::OpticalPhotonDefinition())
33 +     return false;
34 + if (step->GetPreStepPoint()->GetStepStatus() != fGeomBoundary)
35 +     return false;
36
37 - // get time and energy of photons
38 + const int parentId = track->GetParentID();
39 + const double time = step->GetPreStepPoint()->GetGlobalTime() / CLHEP::second;
40 - if (time >= 1*CLHEP::second)
41 -     return true;
42 + if (time >= 1*CLHEP::second) {
43 +     track->SetTrackStatus(fStopAndKill);
44 +     return false;
45 + }
46
47 auto& pmt = fEvent.GetDetector(fDetectorId).GetOptDevice(fOptDeviceId);
48 - double energy = step->GetPreStepPoint()->GetKineticEnergy() / CLHEP::eV;
49 - // kill if photon energy is out of PMT range
50 - if (energy < pmt.GetOpticalRange()[0] || energy > pmt.GetOpticalRange()[1])
51 -     return true;
52 + const double energy = step->GetPreStepPoint()->GetKineticEnergy() / CLHEP::eV;
53 + const std::vector<double> opticalRange = pmt.GetOpticalRange();
54 + if (energy < opticalRange[0] || energy > opticalRange[1]) {
55 +     track->SetTrackStatus(fStopAndKill);
56 +     return false;
57 + }
58
59 SimData& simData = fEvent.GetSimData();
60 - if (simData.GetSimulationMode() == SimData::SimulationMode::eFull) {
61 -

```

```

62 - // kill according to PMT quantum efficiency
63 - if (!pmt.IsPhotonDetected(energy))
64 -     return true;
65 + if (simData.GetSimulationMode() == SimData::SimulationMode::eFull &&
66 +     !pmt.IsPhotonDetected(energy)) {
67 +     track->SetTrackStatus(fStopAndKill);
68 +     return false;
69 + }
70 -
71 -
72 - DetectorSimData& detSimData = simData.GetDetectorSimData();
73 - const auto& muDecayIDs = detSimData.GetMuonDecayID();
74
75 + DetectorSimData& detSimData = simData.GetDetectorSimData(fDetectorId);
76 + const auto& muDecayIDs = detSimData.GetMuonDecayID();
77 + if (muDecayIDs.find(parentId) != muDecayIDs.end()) {
78 -     //cout << "[DEBUG] G4MPMTAction: Photon from MUON DECAY was detected! " << endl;
79     fPETimeMuDecay.push_back(time);
80 -     NumPEMuDecay += 1;
81 +     ++NumPEMuDecay;
82 + }
83
84 - string procName = step->GetTrack()->GetCreatorProcess()->GetProcessName();
85 -
86 -
87 - // add photon time to SimData
88     fPETime.push_back(time);
89     fPETimeComp.push_back(time);
90 -
91 + ++NumPE;
92 +
93 + // Detection is terminal in this effective model. This prevents the same
94 + // photon from generating multiple photoelectrons in later Pyrex steps.
95 + track->SetTrackStatus(fStopAndKill);
96 +     return true;
97
98 }
99
100 -
101 -

```

A.8. Framework/Detector/OptDevice.cc

```

1 --- MEIGA-anterior/src/Framework/Detector/OptDevice.cc
2 +++ MEIGA-integrador/src/Framework/Detector/OptDevice.cc
3 @@ -1,6 +1,7 @@
4 #include "OptDevice.h"
5 #include "CLHEP/Random/RandFlat.h"
6 #include "CLHEP/Random/RandGauss.h"
7 +#include <algorithm>
8
9 using namespace std;
10
11 @@ -137,25 +138,9 @@
12
13     double waveLength = 1240. / energy;
14     DeviceType t = GetType();
15 - double qeff = GetQuantumEfficiency(waveLength, t);
16 - double rand = CLHEP::RandFlat::shoot();
17 -
18 -     std::ofstream outputFileqe2("Datos-simulacion/eficiencia-limpia-2.txt", std::ios_base::app);
19 -     outputFileqe2 << energy << "\t" << waveLength << "\t" << qeff << "\t" << rand << "\n";
20 -     outputFileqe2.close();
21 -
22 -
23 -
24 - //return (rand >= qeff); Forma incorrecta
25 -
26 - return (rand < qeff);
27
28 - //std::ofstream outputFileqe3("Datos-simulacion/eficiencia-limpia-3.txt", std::ios_base::app);
29 - //outputFileqe3 << energy << "\t" << waveLength << "\t" << qeff << "\t" << rand << "\n";
30 - //outputFileqe3.close();

```

```

31 -
32 -
33 -
34 + const double detectionEfficiency =
35 + std::max(0.0, std::min(1.0, GetQuantumEfficiency(waveLength, t)));
36 + return CLHEP::RandFlat::shoot() < detectionEfficiency;
37 }
38
39
40 @@ -207,25 +192,28 @@
41
42     case ePMT:
43     {
44 -
45 -         if(waveLength >= 250. && waveLength < 300.)
46 -             qeff = 0.01;
47 -         else if(waveLength >= 300. && waveLength < 350.)
48 -             qeff = 0.03;
49 -         else if(waveLength >= 350. && waveLength < 400.)
50 -             qeff = 0.2;
51 -         else if(waveLength >= 400. && waveLength < 450.)
52 -             qeff = 0.25;
53 -         else if(waveLength >= 450. && waveLength < 500.)
54 -             qeff = 0.2;
55 -         else if(waveLength >= 500. && waveLength < 550.)
56 -             qeff = 0.14;
57 -         else if(waveLength >= 550. && waveLength < 600.)
58 -             qeff = 0.07;
59 -         else if(waveLength >= 600. && waveLength < 650.)
60 -             qeff = 0.03;
61 -         else if(waveLength >= 650. && waveLength < 700.)
62 -             qeff = 0.06;
63 +         // Nominal R5912 response used by this WCD model. Values between
64 +         // tabulated wavelengths are linearly interpolated, avoiding the
65 +         // artificial jumps of the former 50-nm histogram. The endpoints
66 +         // make the supported 300--650 nm range explicit.
67 +         static const double wavelengthNm[] = {
68 +             300., 350., 400., 450., 500., 550., 600., 650.
69 +         };
70 +         static const double quantumEfficiency[] = {
71 +             0.03, 0.20, 0.25, 0.20, 0.14, 0.07, 0.03, 0.00
72 +         };
73 +         const std::size_t n = sizeof(wavelengthNm) / sizeof(double);
74 +         if (waveLength >= wavelengthNm[0] && waveLength <= wavelengthNm[n-1]) {
75 +             for (std::size_t i = 1; i < n; ++i) {
76 +                 if (waveLength <= wavelengthNm[i]) {
77 +                     const double fraction = (waveLength - wavelengthNm[i-1]) /
78 +                                             (wavelengthNm[i] - wavelengthNm[i-1]);
79 +                     qeff = quantumEfficiency[i-1] + fraction *
80 +                         (quantumEfficiency[i] - quantumEfficiency[i-1]);
81 +                     break;
82 +                 }
83 +             }
84 +         }
85
86     #if 0
87         // photonis-XP1805
88     @@ -294,13 +282,8 @@
89
90         qeff *= fQEScaleParameter * fPMTCollectionEfficiency;
91     #endif
92 -
93 -
94 -     std::ofstream outputFileqe("Datos-simulacion/eficiencia-limpia.txt", std::ios_base::app);
95 -     outputFileqe << waveLength << "\t" << qeff << "\n";
96 -     outputFileqe.close();
97 -
98 -
99 +         // The Bernoulli probability is the photon-detection efficiency:
100 +         // photocathode QE times first-dynode collection efficiency.
101         qeff *= fPMTCollectionEfficiency;
102     }
103     break;
104 @@ -310,12 +293,6 @@

```

```

105     break;
106
107 } // end switch
108 - std::ofstream outputFile1("Datos-simulacion/eficiencia-limpia-1.txt", std::ios_base::app);
109 - outputFile1 << waveLength <<"\t" << qeff <<"\n";
110 - outputFile1.close();
111 -
112 -
113 -
114     return qeff;
115
116 }
117 @@ -336,7 +313,8 @@
118     break;
119
120     case ePMT:
121 -         fOpticalRange = {1.77 ,4.96};
122 +         // 650 nm <= lambda <= 300 nm, expressed as increasing energy.
123 +         fOpticalRange = {1240. / 650., 1240. / 300.};
124     break;
125
126     case eUnknown:

```

B. Archivos de referencia sin cambios

Los siguientes archivos, aportados como copias de referencia en la carpeta **Correcciones-PMT/**, se compararon byte a byte contra el Meiga anterior y resultaron **idénticos**, por lo que no requirieron ninguna acción: **Detector.h**, **Detector.cc**, **OptDevice.h**, **OpticalPhysics.cc**, **OpticalPhysics.hh**, **G4MPMTAction.h** (copia sin sufijo) y **G4MPMTAction.cc** (copia sin sufijo). También **G4Models/SaltyWCD.h** se mantuvo sin cambios: ninguna de las revisiones de **SaltyWCD.cc** añadió miembros nuevos a la clase, por lo que su interfaz pública (**BuildDetector**) permanece igual.